

幻尔科技

ArmPi Ultra 机械臂+滑轨结合课程



Hiwonder 官方网站



版本号	修改日期	修改摘要
V1.0	20250812	初次发布

目录

1. 滑轨安装	1
2. 机器类型选择	8
3. 位置校准	10
4. 手柄控制	10
4.1. 准备工作	10
4.2. 设备连接	11
4.3. 控制模式介绍	12
5. 色块分拣	16
5.1. 玩法简介	错误！未定义书签。
5.2. 玩法步骤	错误！未定义书签。
5.3. 实现效果	17
5.4. 程序分析	17
5.4.1. Launch 文件分析	17
5.4.2. Python 文件分析	18
6. 垃圾分类	27
6.1. 玩法简介	错误！未定义书签。
6.2. 玩法步骤	错误！未定义书签。
6.3. 实现效果	28
6.4. 程序分析	29

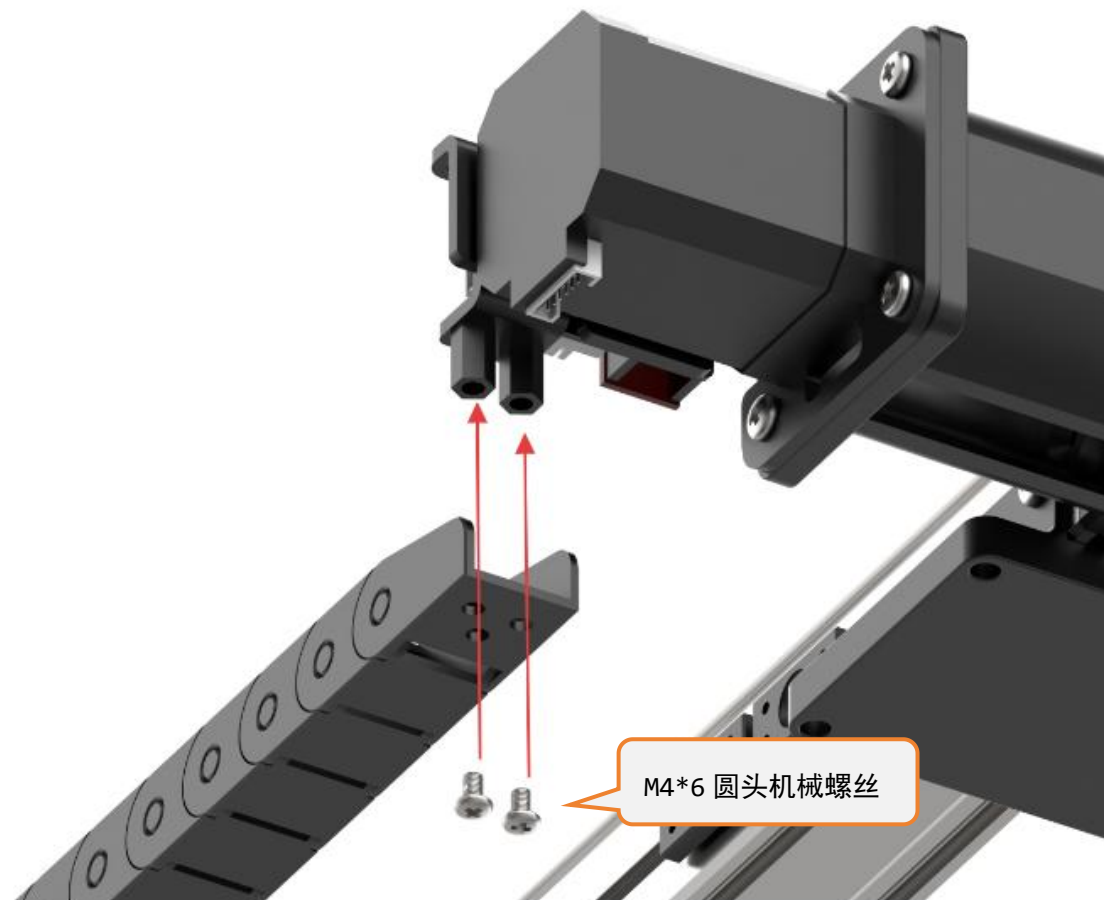
6.4.1. Launch 文件分析	29
6.4.2. Python 文件分析	30
7. 标签分拣	38
7.1. 玩法简介	错误！未定义书签。
7.2. 玩法步骤	错误！未定义书签。
7.3. 实现效果	40
7.4. 程序分析	40
7.4.1. Launch 文件分析	40
7.4.2. Python 文件分析	41

1. 滑轨安装

下文为滑轨与机械臂安装的示意图：

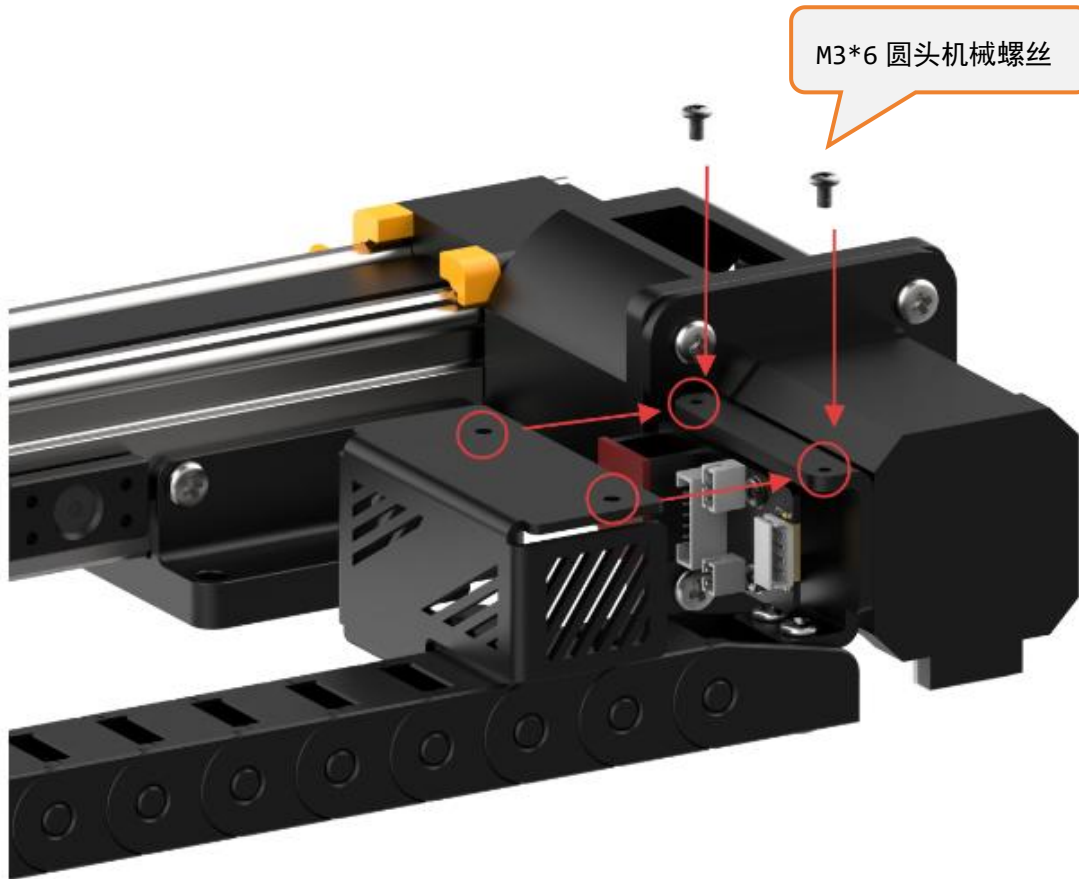
步骤 1：安装坦克带

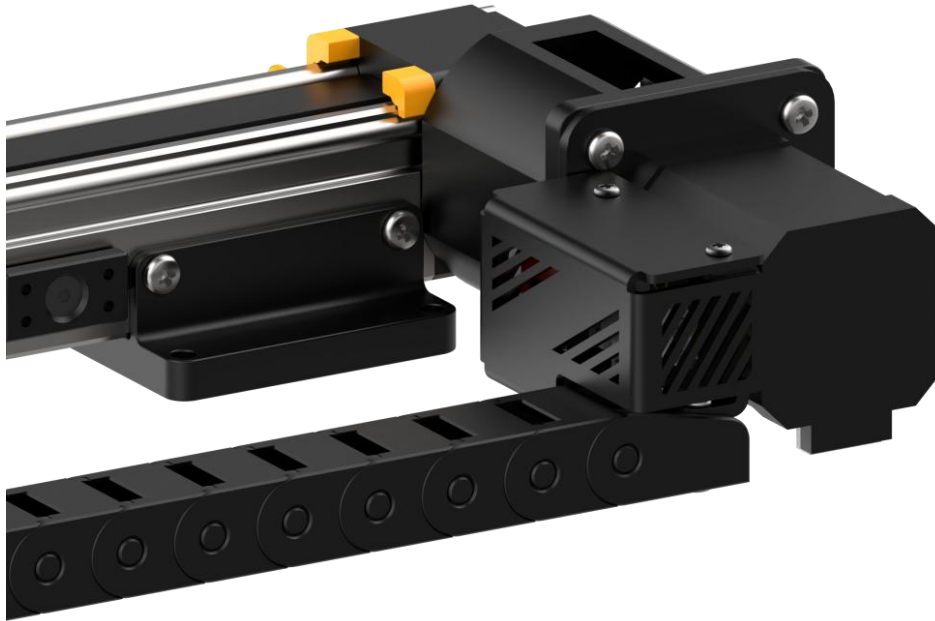
使用 **2 颗 M4*6 圆头机械螺丝** 将坦克带安装在滑轨上。



步骤 2：安装电机固定盖

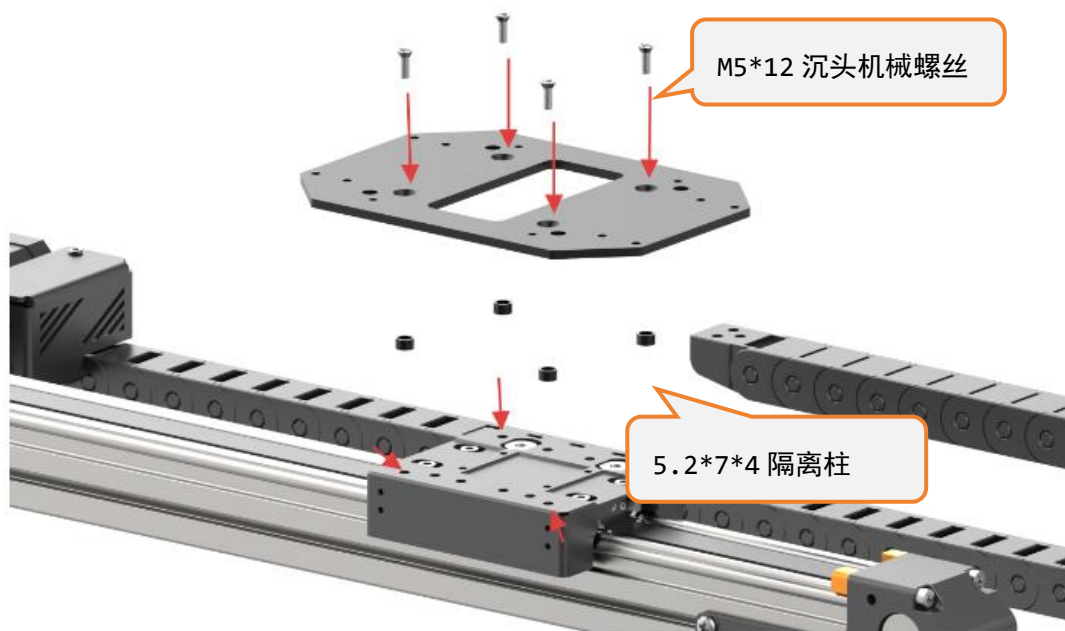
使用 **2 颗 M3*6 圆头机械螺丝**，将电机固定盖安装在滑轨上。

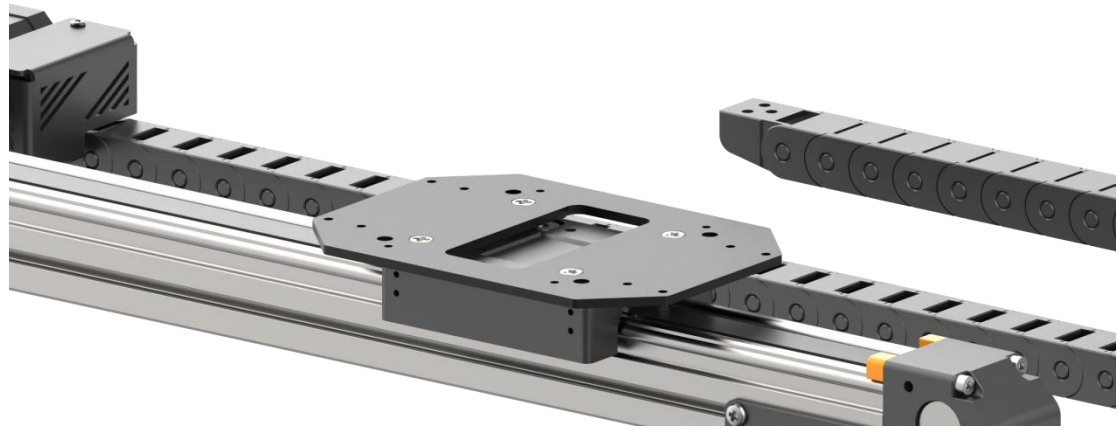




步骤 3：安装滑轨底板

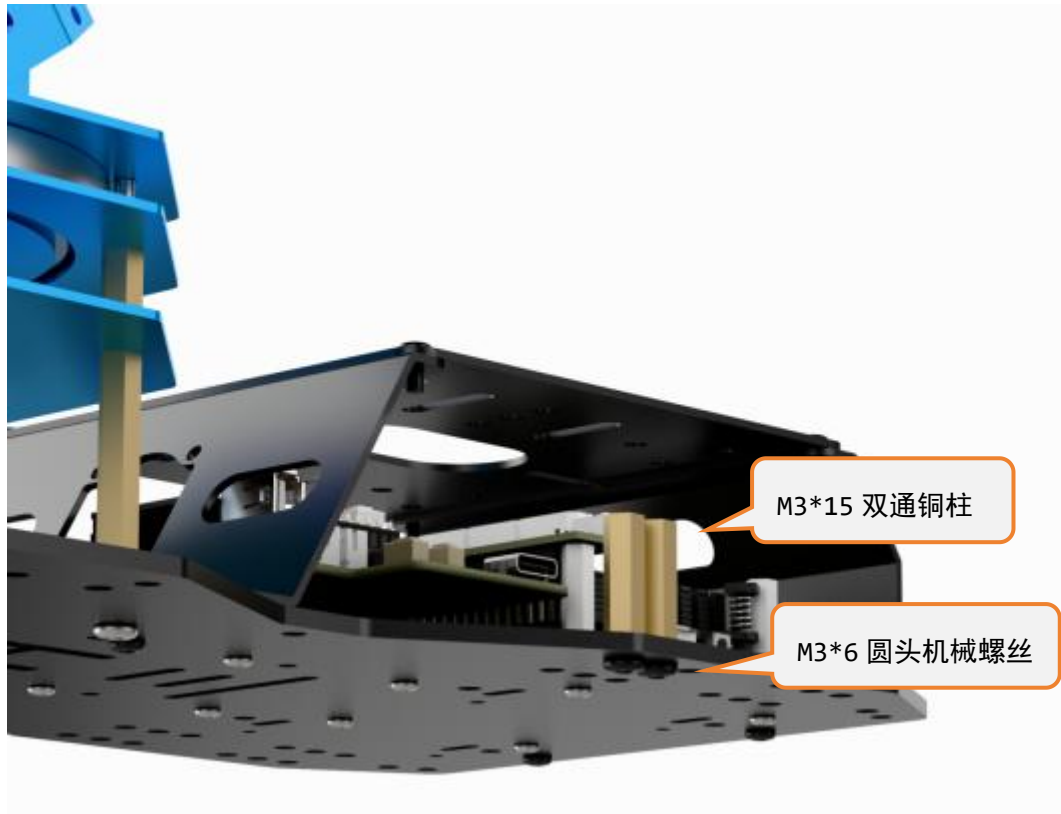
使用 4 颗 M5*12 沉头机械螺丝和 4 个 5.2*7*4 的隔离柱，将滑轨底板固定在滑轨上。





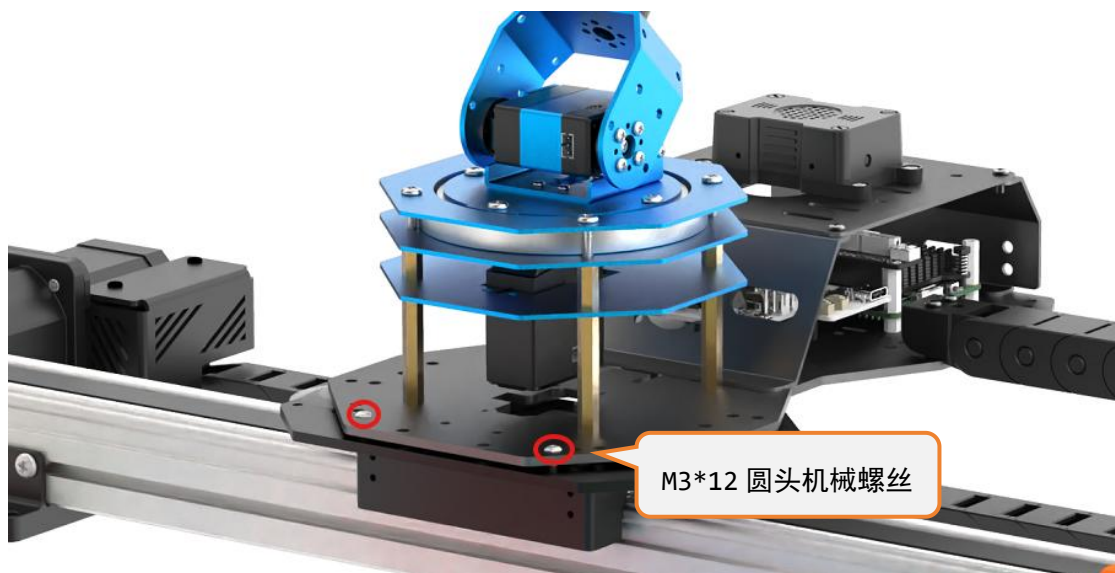
步骤 4：安装铜柱

使用 2 颗 M3*6 圆头机械螺丝将 2 个 M3*15 双通铜柱安装在 ArmPi Ultra 底座上。



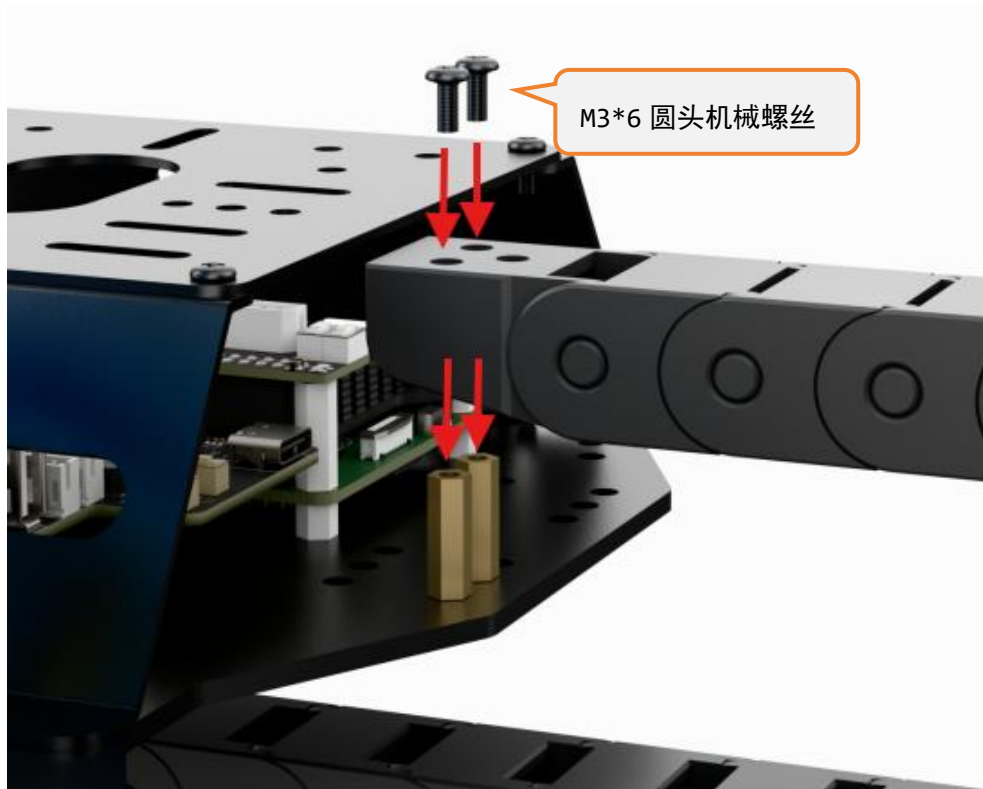
步骤 5：安装机械臂

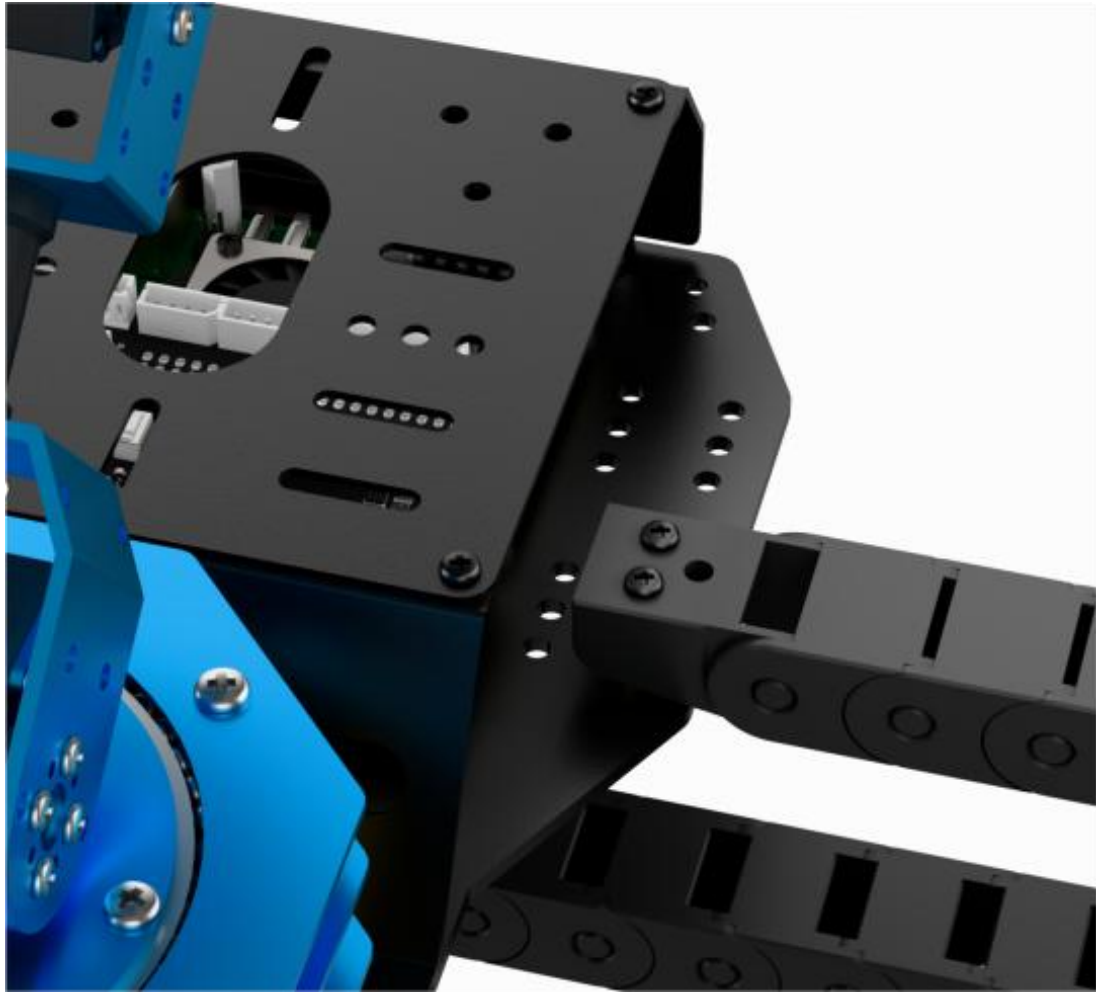
使用 **2 颗 M3*12 圆头机械螺丝** 将机械臂组装到滑轨底板上，并用对应的螺母进行紧固。



步骤 6：坦克带固定在机械臂

使用 2 颗 M3*6 圆头机械螺丝将坦克带组装到 ArmPi Ultra 机械臂上。



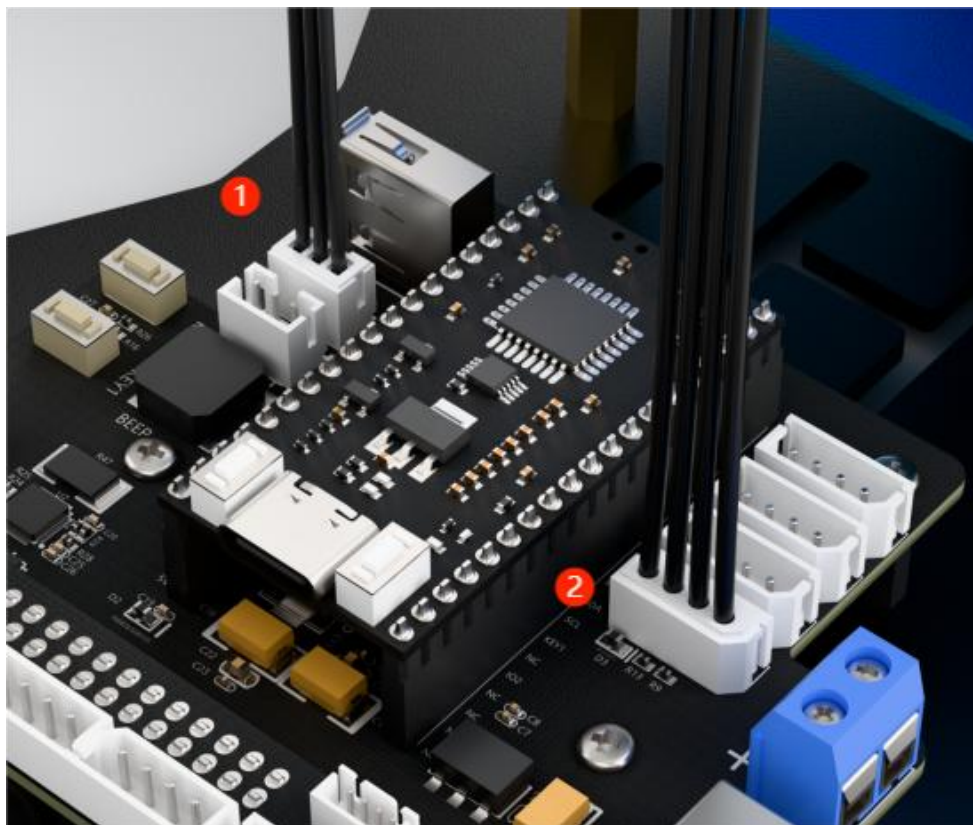


步骤 7：完成效果及接线说明

组装完成效果如下图所示：



滑轨的 3PIN 线接 STM32 扩展板 3PIN 端子口（1 号口）进行供电，4PIN 线接扩展板的 IIC 接口（2 号口）。如下图所示：



2. 机器类型选择

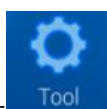
ArmPi Ultra 的拓展配件区分三种：麦轮底盘、滑轨和传送带。在安装过后，要根据

对应的配件切换设备版本之后才能正常使用。

注意：若没有切换或者切换错误，可能造成电机以不可预期的方式运行，导致玩法无法正常运行，甚至损坏设备。

详细步骤如下：

1) 启动机械臂，将其连接至远程控制软件 VNC，关于远程桌面工具的安装与连接可参考“1 教程资料\1 ArmPi Ultra 使用入门\ArmPi Ultra 使用入门手册->开发环境搭建及配置”。



2) 双击打开桌面上的型号配置工具。

3) 根据机械臂的版本、相机版本和配件类型选择合适的选项：



其中，Mecanum 为麦克纳姆轮底盘，Slide_Rails 为滑轨，Conveyor_Belt 为传送带。

4) 选择完成后点击“保存”保存配置。



5) 点击“生效”重新加载配置。



等待蜂鸣器发出“滴”的一声后及重启完成生效。

3. 位置校准

注意：

本节对应的是滑轨形态下的位置校准，请先将机械臂在桌面上调节好机械臂偏差后再将其安装至滑轨来进行操作。

位置校准，即实现机械臂在摄像头回传画面的坐标与实坐标的转换。通过位置校准，可以获取木块与机械臂的实际距离，从而更好地完成分拣等系列玩法效果。

1) 可参考“1.教程资料\1.ArmPi Ultra使用入门\ArmPi Ultra使用入门手册->物品夹取与放置调节”对机械臂的夹取进行调整。

4. 手柄控制

4.1. 准备工作

1. 请在设备开机前确认手柄接收器是否插入，如已插入可忽略（在出厂时，机器人已插入了USB手柄接收器）；

2. 放置电池时，须注意区分电池正负极；



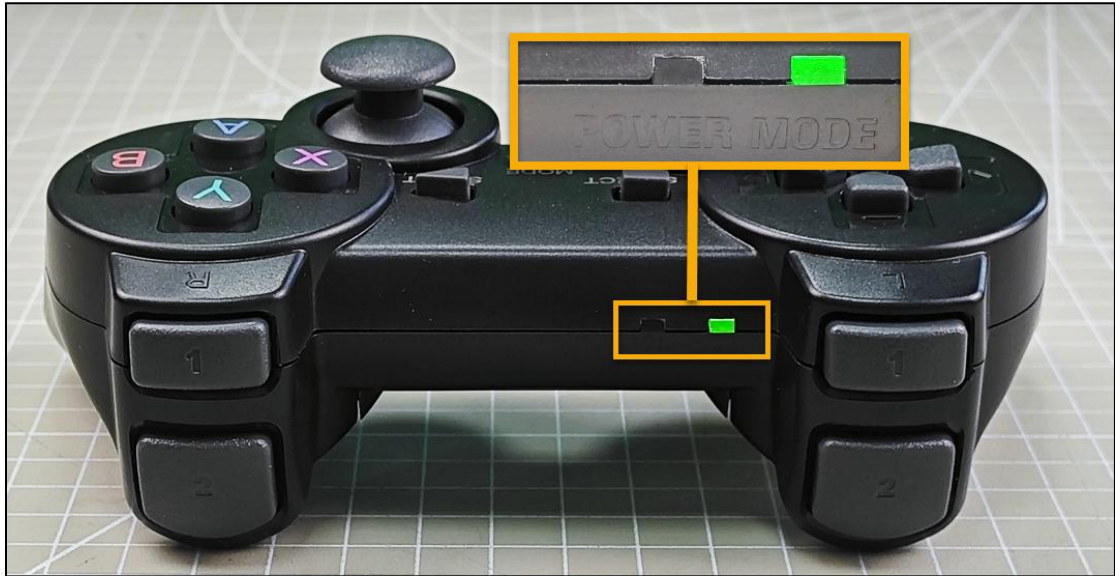
3. 每次开启机器人时，会启动 APP 自启服务 (APP 服务中包含手柄控制服务)，如未关闭则无需另外操作，直接连接控制即可；

4. 由于手柄控制具有互串性，多台机器人在同一场地内，建议不要使用此功能，避免误连误控；

5. 打开手柄开关后，若在 30 秒内未连接机器人，或在连接机器人后 5 分钟内未使用手柄，手柄会自动进入睡眠模式。如需唤醒手柄，即退出睡眠模式，按下“START”键即可。

4.2. 设备连接

1. 机器人启动完成后，将手柄开关推至“ON”挡。此时，手柄上的两个 LED 灯（红色、绿色）会同时闪烁。
2. 等待几秒，机器人和手柄开始自动配对。当配对成功，绿色 LED 灯常亮，红色 LED 灯熄灭。

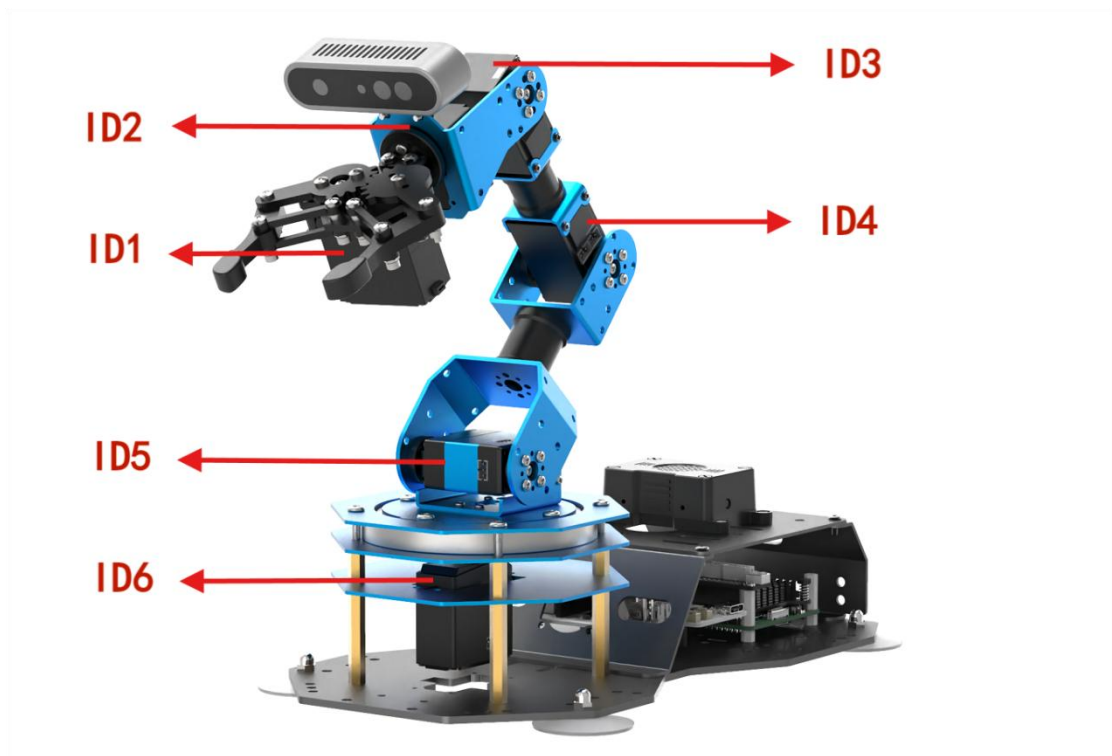


4.3. 控制模式介绍

手柄控制分为两种模式：坐标模式和单舵机模式。手柄连接成功后，默认为单舵机模式。

按下 4 次 “MODE” 按键，在提示音响四声的时候，即为控制滑轨的模式。

◆ 单舵机模式：通过手柄的按键，可控制机械臂上单个舵机进行正反转动。

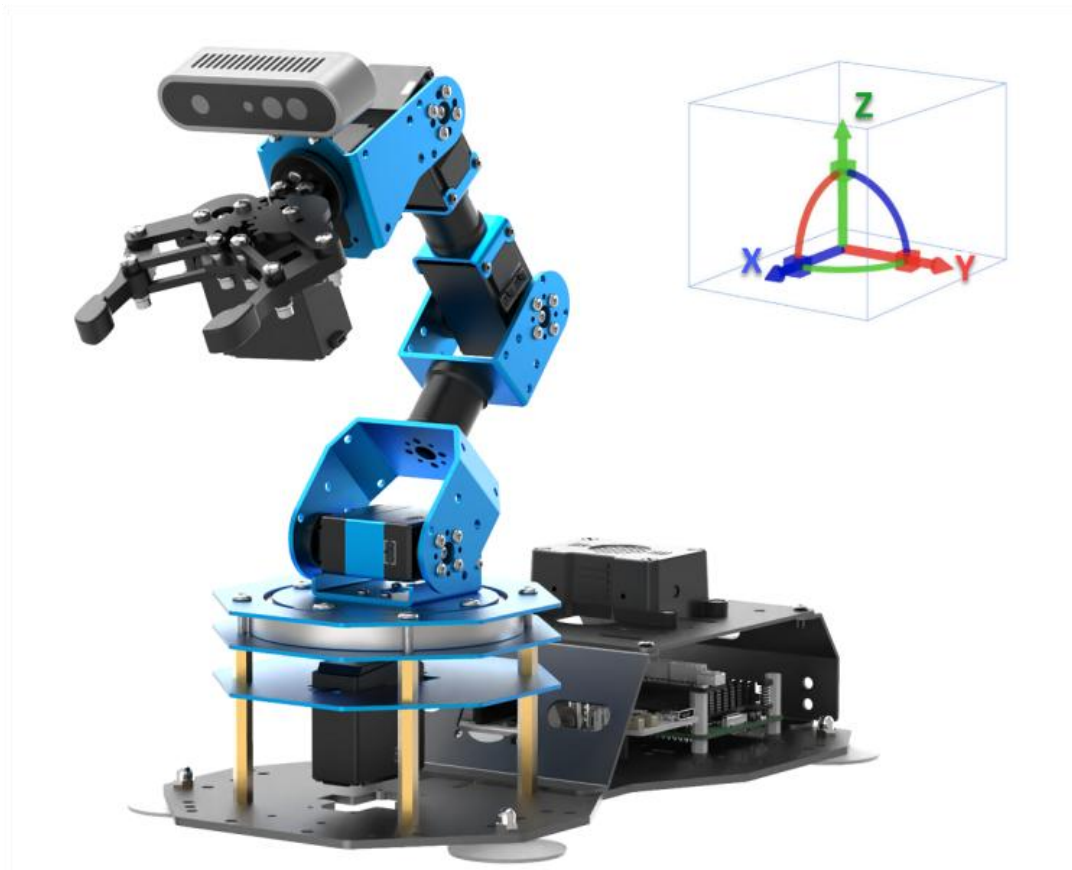


下表为单舵机模式的按键功能说明：

按键	功能（以机械臂为第一视角）
START	机械臂复位
SELECT+START	切换控制模式(单舵机/坐标)
UP / ↑	5 号舵机抬高
DOWN / ↓	5 号舵机降低
LEFT / ←	6 号舵机左转
RIGHT / →	6 号舵机右转
Y	4 号舵机降低
A	4 号舵机抬高
B	2 号舵机右转（即机械爪右转）
X	2 号舵机左转（即机械爪左转）
L1	机械爪张开（1 号舵机）
L2	机械爪闭合（1 号舵机）
R1	3 号舵机抬高
R2	3 号舵机降低
左摇杆 LEFT	滑轨向左移

左摇杆 RIGHT	滑轨向右移
-----------	-------

◆ 坐标模式：通过手柄的按键，可控制机械臂以一个整体沿三轴坐标（x、y、z）的轨迹以及结合偏转角进行运动。



两种模式切换方法：可按下“SELECT”键和“START”键，当听到提示音后即为切换成功。

- ① 单舵机模式切换到坐标模式：响两声；
- ② 坐标模式切换到单舵机模式：响一声。

下表为坐标模式的按键功能说明：

按键	功能（以机械臂为第一视角）
----	---------------

START	机械臂复位
SELECT+START	切换控制模式(单舵机/坐标)
UP / ↑	机械臂向 X 轴正方向移动（机械臂前方）
DOWN / ↓	机械臂向 X 轴负方向移动（机械臂后方）
LEFT / ←	机械臂向 Y 轴正方向移动（机械臂左侧）
RIGHT / →	机械臂向 Y 轴负方向移动（机械臂右侧）
Y	机械爪闭合（1 号舵机）
A	机械爪张开（1 号舵机）
B	2 号舵机右转（即机械爪右转）
X	2 号舵机左转（即机械爪左转）
L1	机械臂向 Z 轴正方向移动（机械臂上方）
L2	机械臂向 Z 轴负方向移动（机械臂下方）
R1	增大机械爪俯仰角度
R2	减小机械爪俯仰角度
左摇杆 LEFT	滑轨向左移
左摇杆 RIGHT	滑轨向右移

5. 色块分拣

5.1. 玩法简介

本节通过摄像头使用 Opencv 算法识别目标颜色的色块，计算出色块的中心位置。之后，调用逆运动学算法计算出机械臂舵机的脉冲宽度，驱动机械臂夹取目标色块。最后，根据目标色块的放置地点移动滑轨，之后执行放置动作，完成色块分拣功能。

5.2. 玩法步骤

注意：指令的输入需严格区分大小写，另外可按键盘“Tab”键进行关键词补齐。

1) 参照“1.教程资料\1.ArmPi Ultra 使用入门\ArmPi Ultra 使用入门手册->开发环境搭建及配置”将机械臂与远程桌面工具建立连接。

2) 单击 ，启动命令栏，输入指令，按下回车，关闭自启动服务。

```
~/ .stop_ros.sh
```

```
> ~/ .stop_ros.sh
```

3) 输入运行色块分拣的指令，按下回车。

```
ros2 launch example color_sorting_stepper.launch.py
```

```
ros2 launch example color_sorting_stepper.launch.py
```

4) 如需关闭此程序，可按下“Ctrl+C”，若玩法关闭失败可重复多次按下。

5) 玩法体验完成后，需开启 APP 服务 （**若不开启则会影响后续 APP 玩法的使用**）。在终端界面输入按下回车，启动 APP 服务。

```
ros2 launch bringup bringup.launch.py
```

```
> ros2 launch bringup bringup.launch.py
```

6) APP 服务启动完成后机械臂会回到初始姿态，蜂鸣器也会“滴”一声。

5.3. 实现效果

开启玩法后, 将目标色块放置在机械臂的视觉识别区, 机械臂将逐个对色块进行抓取, 并放置到对应的位置。

5.4. 程序分析

5.4.1. Launch 文件分析

launch 文件位于:

`~/ros2_ws/src/example/example/stepper/color_sorting_stepper.launch.p`

y

◆ launch_setup 函数

```
def launch_setup(context):
    compiled = os.environ['need_compile']
    if compiled == 'True':
        sdk_package_path = get_package_share_directory('sdk')
        peripherals_package_path = get_package_share_directory('peripherals')
    else:
        sdk_package_path = '/home/ubuntu/ros2_ws/src/driver/sdk'
        peripherals_package_path = '/home/ubuntu/ros2_ws/src/peripherals'

    depth_camera_launch = IncludeLaunchDescription(
        PythonLaunchDescriptionSource(
            os.path.join(peripherals_package_path, 'launch/depth_camera.launch.py')),
    )

    sdk_launch = IncludeLaunchDescription(
        PythonLaunchDescriptionSource(
            os.path.join(sdk_package_path, 'launch/arduino_ultra.launch.py')),
    )

    color_sorting_stepper_node = Node(
        package='example',
        executable='color_sorting_stepper',
        output='screen',
    )

    return [
        sdk_launch,
        depth_camera_launch,
        color_sorting_stepper_node,
    ]
```

加载 peripherals 功能包的 launch/depth_camera.launch.py, 启动深度相机节点, 提供 RGB 图像和深度数据, 作为颜色识别的视觉输入。加载 sdk 功能包的 launch/arm_pi_ultra.launch.py, 启动机械臂底层控制服务(如舵机驱动、运动学计算), 为分拣动作提供硬件控制支持。启动 example 功能包的 color_sorting_stepper 可执行文件, 结合视觉识别结果, 控制滑轨, 完成颜色分拣的自动化流程, 日志输出到屏幕。

◆ generate_launch_description 函数

```
def generate_launch_description():  
    return LaunchDescription([  
        OpaqueFunction(function = launch_setup)  
    ])
```

创建并返回 LaunchDescription 对象, 通过 OpaqueFunction 调用 launch_setup 函数, 作为 ROS 2 启动文件的标准入口。

◆ 主函数

```
if __name__ == '__main__':  
    ld = generate_launch_description()  
  
    ls = LaunchService()  
    ls.include_launch_description(ld)  
    ls.run()
```

创建 LaunchDescription 对象和 LaunchService 服务, 将启动描述添加到启动服务并运行, 实现手动启动整个系统的功能。

5.4.2. Python 文件分析

python 文件位于:

~/ros2_ws/src/example/example/stepper/include/color_sorting_stepper.py

◆ 导入功能包

```
import os
import cv2
import yaml
import time
import math
import queue
import rclpy
import threading
import numpy as np
from sdk import common
from rclpy.node import Node
from cv_bridge import CvBridge
from interfaces.srv import SetStringBool
from std_srvs.srv import Trigger, SetBool
from sensor_msgs.msg import Image, CameraInfo
from rclpy.executors import MultiThreadedExecutor
from servo_controller_msgs.msg import ServosPosition
from rclpy.callback_groups import ReentrantCallbackGroup
from kinematics.kinematics_control import set_pose_target
from kinematics_msgs.srv import GetRobotPose, SetRobotPose
from servo_controller.bus_servo_control import set_servo_position
from ros_robot_controller_msgs.srv import MoveStepper
from app.utils import calculate_grasp_yaw, position_change_detect, pick_and_place, image_process, distortion_inverse_map
```

- ① **sdk.common** : 这些模块是机械臂 SDK（软件开发工具包）的一部分。
- ② **servo_controller_msgs.msg.ServosPosition**: 用于控制伺服电机位置的消息类型，通常用于实现伺服控制。
- ③ **kinematics.kinematics_control.set_pose_target**: 用于设置机器人的目标位姿（位置和朝向），依赖于运动学计算。
- ④ **kinematics_msgs.srv.GetRobotPose** 和 **SetRobotPose**: 服务接口，用于获取和设置机器人的位姿信息。
- ⑤ **servo_controller.bus_servo_control.set_servo_position**: 用于设置伺服电机位置的函数，主要用于控制机器人的抓取或移动操作。

◆ 指定目标位置

```
TARGET_POSITION = {
    'red': (2600),
    'green': (4000),
    'blue': (5200),
}
```

这是一个字典，用于存储不同颜色（红色、绿色和蓝色）的目标位置。这个结构可以用来简化在机器人任务中根据颜色快速查找相应的位置。滑轨位置范围为 0(最左端)~5200(最右端)。

◆ ColorSortingNode 类初始化

```
def __init__(self, name):
    rclpy.init()
    super().__init__(name, allow undeclared parameters=True, automatically declare parameters from overrides=True)

    self.K = None
    self.D = None
    self.lock = threading.RLock()
    self.image_queue = queue.Queue(maxsize=2)
    self.bridge = CvBridge()
    self.imgpts = None
    self.tag_size = 0.025
    self.center_imgpts = None
    self.roi = None
    self.count_move = 0
    self.count_still = 0
    self.target_miss_count = 0
    self.white_area_width = 0.167
    self.white_area_height = 0.108
    self.calibration_file = 'calibration.yaml'
    self.config_file = 'transform.yaml'
    self.config_path = "/home/ubuntu/ros2_ws/src/example/example/stepper/config/"
    self.data = common.get_yaml_data("/home/ubuntu/ros2_ws/src/app/config/lab_config.yaml")
    self.lab_data = self.data['/**']['ros_parameters']
    self.camera_type = os.environ['CAMERA_TYPE']
    self.min_area = 500
    self.max_area = 7000
    self.target = None
    self.last_position = None
    self.start_transport = False

    self.image_sub = None
    self.camera_info_sub = None
    self.joints_pub = self.create_publisher(ServosPosition, '/servo_controller', 1)
    self.image_sub = self.create_subscription(Image, '/depth_cam/rgb/image_raw', self.image_callback, 1)
    self.camera_info_sub = self.create_subscription(CameraInfo, '/depth_cam/rgb/camera_info', self.camera_info_callback, 1)

    self.timer_cb_group = ReentrantCallbackGroup()
    self.client = self.create_client(Trigger, '/controller_manager/init_finish', callback_group=self.timer_cb_group)
    self.client.wait_for_service()
    self.get_current_pose_client = self.create_client(GetRobotPose, '/kinematics/get_current_pose', callback_group=self.timer_cb_group)
    self.get_current_pose_client.wait_for_service()
    self.kinematics_client = self.create_client(SetRobotPose, '/kinematics/set_pose_target', callback_group=self.timer_cb_group)
    self.kinematics_client.wait_for_service()
    self.move_stepper_client = self.create_client(MoveStepper, '/ros_robot_controller/move_stepper', callback_group=self.timer_cb_group)
    self.move_stepper_client.wait_for_service()

    self.timer = self.create_timer(0.0, self.init_process, callback_group=self.timer_cb_group)
```

初始化节点，设置了图像处理相关的订阅和发布，配置了步进电机的控制，并准备了与机器人运动学相关的服务客户端。通过加载 LAB 颜色空间配置文件获取颜色范围参数，创建图像队列进行缓存管理，并通过多线程的方式使主逻辑能够并行运行，实现图像处理与运动控制的协调工作。

◆ send_request 函数

```
def send_request(self, client, msg):
    future = client.call_async(msg)
    while rclpy.ok():
        if future.done() and future.result():
            return future.result()
```

封装服务调用逻辑，发送请求后阻塞等待结果，确保机械臂运动学、步进电机等服务的通信可靠性。

◆ camera_info_callback 函数

```
def camera_info_callback(self, msg):  
    K = np.matrix(msg.k).reshape(1, -1, 3)  
    D = np.array(msg.d)  
    new_K, roi = cv2.getOptimalNewCameraMatrix(K, D, (640, 400), 0, (640, 400))  
    self.K, self.D = np.matrix(new_K), np.zeros((5, 1))
```

接收相机内参消息，提取相机矩阵（K）和畸变系数（D），并通过 cv2.getOptimalNewCameraMatrix 优化内参，为后续坐标转换提供基础。

◆ adaptive_threshold 函数

```
def adaptive_threshold(self, gray_image):  
    binary = cv2.adaptiveThreshold(gray_image, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C, cv2.THRESH_BINARY, 41, 7)  
    return binary
```

对灰度图像进行自适应高斯阈值处理，生成二值图像，增强物体边缘特征。

◆ canny_proc 函数

```
def canny_proc(self, bgr_image):  
    mask = cv2.Canny(bgr_image, 9, 41, 9, L2gradient=True)  
    mask = 255 - cv2.dilate(mask, cv2.getStructuringElement(cv2.MORPH_RECT, (11, 11)))  
    return mask
```

使用 Canny 边缘检测提取图像边缘，通过膨胀操作优化边缘，生成边缘掩码。

◆ get_top_surface 函数

```
def get_top_surface(self, rgb_image):  
    image_scale = cv2.convertScaleAbs(rgb_image, alpha=2.5, beta=0)  
    image_gray = cv2.cvtColor(image_scale, cv2.COLOR_RGB2GRAY)  
    image_mb = cv2.medianBlur(image_gray, 3)  
    image_gs = cv2.GaussianBlur(image_mb, (5, 5), 5)  
    binary = self.adaptive_threshold(image_gs)  
    mask = self.canny_proc(image_gs)  
    mask1 = cv2.bitwise_and(binary, mask)  
    roi_image_mask = cv2.bitwise_and(rgb_image, rgb_image, mask=mask1)  
    return roi_image_mask
```

结合图像缩放、灰度化、模糊、阈值处理和边缘检测，提取物体顶面区域，减少背景干扰。

◆ image_callback 函数

```
def image_callback(self, ros_image):
    cv_image = self.bridge.imgmsg_to_cv2(ros_image, "rgb8")
    rgb_image = np.array(cv_image, dtype=np.uint8)
    if self.image_queue.full():
        self.image_queue.get()
    self.image_queue.put(rgb_image)
```

将 ROS 图像消息 (sensor_msgs/Image) 转换为 OpenCV 格式 (RGB)，存入图像队列 (队列满时丢弃旧图像)，供后续处理。

◆ init_process 函数

```
def init_process(self):
    self.timer.cancel()
    req = MoveStepper.Request()
    req.steps = 0
    self.send_request(self.move_stepper_client, req)
    set_servo_position(self.joints_pub, 1.5, ((1, 210), (2, 500), (3, 110), (4, 825), (5, 600), (6, 500)))
    time.sleep(1.5)
    threading.Thread(target=self.main, daemon=True).start()
    threading.Thread(target=self.transport_thread, daemon=True).start()
    with open(self.config_path + self.config_file, 'r') as f:
        config = yaml.safe_load(f)
        corners = np.array(config['corners']).reshape(-1, 3)
        extrinsic = np.array(config['extrinsic'])
        white_area_pose_world = np.array(config['white_area_pose_world'])

    self.white_area_center = white_area_pose_world.reshape(4, 4)
    tvec = extrinsic[:1]
    rmat = extrinsic[1:]
    while self.K is None or self.D is None:
        time.sleep(0.5)

    center_imgpts, jac = cv2.projectPoints(corners[-1:], np.array(rmat), np.array(tvec), self.K, self.D)
    self.center_imgpts = np.int32(center_imgpts).reshape(2)

    tvec, rmat = common.extrinsic_plane_shift(np.array(tvec).reshape((3, 1)), np.array(rmat), 0.03)
    self.extrinsic = tvec, rmat
    tvec, rmat = common.extrinsic_plane_shift(np.array(tvec).reshape((3, 1)), np.array(rmat), 0.0)
    imgpts, jac = cv2.projectPoints(corners[:-1], np.array(rmat), np.array(tvec), self.K, self.D)
    self.imgpts = np.int32(imgpts).reshape(-1, 2)
    x_min = min(self.imgpts, key=lambda p: p[0])[0]
    x_max = max(self.imgpts, key=lambda p: p[0])[0]
    y_min = min(self.imgpts, key=lambda p: p[1])[1]
    y_max = max(self.imgpts, key=lambda p: p[1])[1]
    roi = np.maximum(np.array([y_min, y_max, x_min, x_max]), 0)
    self.roi = roi
```

取消初始化定时器，复位步进电机，将机械臂舵机调整到初始姿态（等待 1.5 秒确保到位）。启动主循环线程 (main) 和运输控制线程 (transport_thread)。加载配置文件 (transform.yaml)，解析校准参数（如角落坐标、外参、白色区域位姿），计算图像 ROI（感兴趣区域），用于限制识别范围。

◆ image_processing 函数

```

def image_processing(self):
    rgb_image = self.image_queue.get()
    result_image = np.copy(rgb_image)
    if self.center_imgpts is not None:
        cv2.line(result_image, (self.center_imgpts[0]-10, self.center_imgpts[1]), (self.center_imgpts[0]+10, self.center_imgpts[1]), (255, 255, 0), 2)
        cv2.line(result_image, (self.center_imgpts[0], self.center_imgpts[1]-10), (self.center_imgpts[0], self.center_imgpts[1]+10), (255, 255, 0), 2)
    target_list = []
    index = 0
    if self.roi is not None and self.imgpts is not None and not self.start_transport:
        roi_area_mask = np.zeros(shape=(rgb_image.shape[0], rgb_image.shape[1], 1), dtype=np.uint8)
        roi_area_mask = cv2.drawContours(roi_area_mask, [self.imgpts], -1, 255, cv2.FILLED)
        rgb_image = cv2.bitwise_and(rgb_image, rgb_image, mask=roi_area_mask)
        roi_img = rgb_image[self.roi[0]:self.roi[1], self.roi[2]:self.roi[3]]
        roi_img = self.get_top_surface(roi_img)
        image_lab = cv2.cvtColor(roi_img, cv2.COLOR_RGB2LAB)
        img_h, img_w = rgb_image.shape[:2]

        for i in ['red', 'green', 'blue']:
            mask = cv2.inRange(image_lab, tuple(self.lab_data['color_range_list'][i]['min']), tuple(self.lab_data['color_range_list'][i]['max']))
            eroded = cv2.erode(mask, cv2.getStructuringElement(cv2.MORPH_RECT, (3, 3)))
            dilated = cv2.dilate(eroded, cv2.getStructuringElement(cv2.MORPH_RECT, (3, 3)))
            contours = cv2.findContours(dilated, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_NONE)[-2]
            contours_area = map(lambda c: (math.fabs(cv2.contourArea(c)), c), contours)
            contours = map(lambda a_c: a_c[1], filter(lambda a: self.min_area <= a[0] <= self.max_area, contours_area))
            for c in contours:
                rect = cv2.minAreaRect(c)
                (center_x, center_y), _ = cv2.minEnclosingCircle(c)
                center_x, center_y = self.roi[2] + center_x, self.roi[0] + center_y
                cv2.circle(result_image, (int(center_x), int(center_y)), 8, (0, 0, 0), -1)
                corners = list(map(lambda p: (self.roi[2] + p[0], self.roi[0] + p[1]), cv2.boxPoints(rect)))
                cv2.drawContours(result_image, [np.intp(corners)], -1, (0, 255, 255), 2, cv2.LINE_AA)
                index += 1
                angle = int(round(rect[2]))
                target_list.append([i, index, (center_x, center_y), (int(rect[1][0]), int(rect[1][1])), angle])
            cv2.drawContours(result_image, [self.imgpts], -1, (255, 255, 0), 2, cv2.LINE_AA)
            for p in self.imgpts:
                cv2.circle(result_image, tuple(p), 8, (255, 0, 0), -1)
            pass

    return target_list, result_image

```

从队列获取图像，在 ROI 区域内进行处理（通过掩码限制识别范围）。将图像转换为 LAB 颜色空间，使用颜色阈值（从 lab_config.yaml 加载）检测红、绿、蓝物体。对每个颜色物体提取轮廓，计算面积、中心坐标、角度等特征，过滤面积过小或过大的目标。在图像上标记目标（中心、轮廓），返回目标列表（含颜色、坐标、角度）和标记后的图像。

◆ get_object_world_position 函数

```

def get_object_world_position(self, position, intrinsic, extrinsic, white_area_center, height=0.03):
    self.get_logger().info(f"原始位置: {position}") # 打印物体的初始位置
    projection_matrix = np.row_stack((np.column_stack((extrinsic[1], extrinsic[0])), np.array([[0, 0, 0, 1]])))
    world_pose = common.pixels_to_world([position], intrinsic, projection_matrix)[0]
    self.get_logger().info(f"转换后的世界坐标: {world_pose}") # 打印转换后的世界坐标
    world_pose[0] = -world_pose[0]
    world_pose[1] = -world_pose[1]
    position = white_area_center[:3, 3] + world_pose
    position[2] = height

    config_data = common.get_yaml_data(os.path.join(self.config_path, self.calibration_file))
    offset = tuple(config_data['pixel']['offset'])
    scale = tuple(config_data['pixel']['scale'])
    for i in range(3):
        position[i] = position[i] * scale[i]
        position[i] = position[i] + offset[i]
    return position, projection_matrix

```

结合相机内参 (K) 和外参 (extrinsic)，通过投影矩阵计算世界坐标。应用校准文件 (calibration.yaml) 中的缩放因子和偏移量，优化坐标精度，最终返回物体在世界坐标系中的位置。

◆ calculate_pick_grasp_yaw 函数

```
def calculate_pick_grasp_yaw(self, position, target, target_info, intrinsic, projection_matrix):
    yaw = math.degrees(math.atan2(position[1], position[0]))
    self.get_logger().info(f"计算的角度 (yaw): {yaw}") # 打印计算的角度
    if position[0] < 0 and position[1] < 0:
        yaw = yaw + 180
        self.get_logger().info(f"调整后的角度 (yaw): {yaw}") # 打印调整后的角度
    elif position[0] < 0 and position[1] > 0:
        yaw = yaw - 180
    gripper_size = [common.calculate_pixel_length(0.09, intrinsic, projection_matrix),
                    common.calculate_pixel_length(0.02, intrinsic, projection_matrix)]
    return calculate_grasp_yaw.calculate_gripper_yaw_angle(target, target_info, gripper_size, yaw)
```

根据物体世界坐标计算抓取角度 (yaw)，结合 gripper 尺寸和目标姿态，优化抓取方向。

◆ calculate_place_grasp_yaw 函数

```
def calculate_place_grasp_yaw(self, position, angle=0):
    yaw = math.degrees(math.atan2(position[1], position[0]))
    if position[0] < 0 and position[1] < 0:
        yaw = yaw + 180
    elif position[0] < 0 and position[1] > 0:
        yaw = yaw - 180
    yaw1 = yaw + angle
    if yaw < 0:
        yaw2 = yaw1 + 90
    else:
        yaw2 = yaw1 - 90
    yaw = yaw2
    if abs(yaw1) < abs(yaw2):
        yaw = yaw1
    yaw = 500 + int(yaw / 240 * 1000)
    return yaw
```

计算放置时的角度，确保物体正确放置到目标位置。

◆ transport_thread 函数

```
def transport_thread(self):
    while True:
        if self.start_transport:
            position, yaw, target = self.transport_info
            if position[0] > 0.22:
                position[2] += 0.01
            config_data = common.get_yaml_data(os.path.join(self.config_path, self.calibration_file))
            offset = tuple(config_data['kinematics']['offset'])
            scale = tuple(config_data['kinematics']['scale'])
            for i in range(3):
                position[i] = position[i] * scale[i]
                position[i] = position[i] + offset[i]
            finish = pick_and_place.pick(position, 80, yaw, 550, 0.02, self.joints_pub, self.kinematics_client)
            if finish:
                stepper_position = TARGET_POSITION[self.target[0]]
                req = MoveStepper.Request() # 正确创建 MoveStepper 请求对象
                req.steps = stepper_position # 设置步进电机的步数

                # 发送请求给 MoveStepper 服务
                self.send_request(self.move_stepper_client, req)
                stepper_time = stepper_position / 1000 # 计算所需时间
                time.sleep(stepper_time)

                # 继续后续操作
                position = [0.25, 0, 0.015]
                yaw = self.calculate_place_grasp_yaw(position, 0)
                config_data = common.get_yaml_data(os.path.join(self.config_path, self.calibration_file))
                offset = tuple(config_data['kinematics']['offset'])
                scale = tuple(config_data['kinematics']['scale'])
                angle = math.degrees(math.atan2(position[1], position[0]))
                if angle > 45:
                    position = [position[0] * scale[1], position[1] * scale[0], position[2] * scale[2]]
                    position = [position[0] + offset[1], position[1] + offset[0], position[2] + offset[2]]
                elif angle < -45:
                    position = [position[0] * scale[1], position[1] * scale[0], position[2] * scale[2]]
                    position = [position[0] + offset[1], position[1] - offset[0], position[2] + offset[2]]
                else:
                    position = [position[0] * scale[0], position[1] * scale[1], position[2] * scale[2]]
                    position = [position[0] + offset[0], position[1] + offset[1], position[2] + offset[2]]
                finish = pick_and_place.place(position, 80, 500, 200, self.joints_pub, self.kinematics_client)
                if finish:
                    set_servo_position(self.joints_pub, 1.5, ((1, 210), (2, 500), (3, 110), (4, 825), (5, 600), (6, 500)))
                    time.sleep(1.5)
                    # 创建新的请求并发送
                    req = MoveStepper.Request() # 正确创建 MoveStepper 请求对象
                    req.steps = -stepper_position # 让步进电机回到原始位置
                    self.send_request(self.move_stepper_client, req)
                    stepper_time = stepper_position / 1000 # 计算所需时间
                    time.sleep(stepper_time)
                    self.target = None
                    self.start_transport = False
            else:
                time.sleep(0.1)
```

当 start_transport 为 True 时，执行抓取 - 运输 - 放置流程：调用 pick_and_place.pick 控制机械臂抓取物体。通过步进电机服务（move_stepper）将物体运输到目标位置（根据颜色从 TARGET_POSITION 获取步数）。调用 pick_and_place.place 控制机械臂放置物体。步进电机复位，机械臂回到初始姿态，重置状态变量。

◆ main 方法

```
def main(self):
    while True:
        target_list, result_image = self.image_processing()
        if target_list:
            target_miss = True
            for target in target_list:
                if self.target is not None :
                    if self.target[0] != target[0] or self.target[1] != target[1]:
                        continue
                    else:
                        target_miss = False
                        self.target = target
                position, projection_matrix = self.get_object_world_position(target[2], self.K, self.extristic, self.white_area_center, 0.03)
                result = self.calculate_pick_grasp_yaw(position, target, target_list, self.K, projection_matrix)
                if result is not None and self.target is None:
                    self.target = target
                    break
            if self.last_position is not None and self.target is not None :
                e_distance = round(math.sqrt(pow(self.last_position[0] - position[0], 2)) + math.sqrt(pow(self.last_position[1] - position[1], 2)), 5)
                if e_distance <= 0.003:
                    cv2.line(result_image, result[1][0], result[1][1], (255, 255, 0), 2, cv2.LINE_AA)
                    self.count_move = 0
                    self.count_still += 1
                else:
                    self.count_move += 1
                    self.count_still = 0
                if self.count_move > 5:
                    self.target = None
                if self.count_still > 15:
                    self.count_still = 0
                    self.count_move = 0
                    self.target = target
                    yaw = 500 + int(result[0] / 240 * 1000)
                    self.transport_info = [position, yaw, target]
                    self.start_transport = True
                    self.last_position = position
            if target_miss:
                self.target_miss_count += 1
            if self.target_miss_count > 5:
                self.target_miss_count = 0
                self.target = None
        if result_image is not None :
            result_image = cv2.cvtColor(result_image, cv2.COLOR_BGR2RGB)
            cv2.imshow('result_image', result_image)
            key = cv2.waitKey(1)
```

持续调用 `image_processing` 获取目标列表，跟踪目标状态。若目标位置稳定（连续 15 帧位移小于 0.003 米），触发运输流程（`start_transport = True`）。若目标丢失超过 5 帧，重置目标状态。实时显示处理后的图像，支持键盘交互（通过 `cv2.waitKey`）。

◆ 主函数

```
def main():
    node = ColorSortingNode('color_sorting')
    executor = MultiThreadedExecutor()
    executor.add_node(node)
    executor.spin()
    node.destroy_node()
    rclpy.shutdown()
```

启动和运行 ROS2 节点的入口函数。创建 `ColorSortingNode` 实例，使用 `MultiThreadedExecutor` 支持多线程处理，启动节点运行并在结束时正确清理资源。

6. 垃圾分类

6.1. 玩法简介

本节通过摄像头识别训练好的 yolov8 模型，对垃圾木块进行分类，然后通过逆运动学求解控制机械臂抓取垃圾木块，再分拣放置对应的垃圾桶中。

步骤一：识别分类

首先，我们订阅摄像头节点发布的实时画面数据，并将其转换为 Numpy 格式。转换格式后，将图像输入到 yolov8 网络进行缩放、转置、阵列扩展等操作获得处理结果。

将获得的垃圾卡片图像坐标转换为原始图像坐标。转换坐标后，根据识别得到的名称，将垃圾按照设置的规则进行分类，并获取对应的垃圾类别名。然后，将垃圾框出并打印其名称及识别置信度。

步骤二：抓取放置

结果的可靠性，对象的确认需要进行多次重复识别。将图像中置信度最高的垃圾卡片作为第一个分拣对象，为了保证结果的可靠性，对象的确认需要进行多次重复识别。确认目标之后，启动机械臂的分类搬运进程，将目标卡片的像素坐标转换为世界坐标。

控制机械臂运动到垃圾卡片处，让其抓取垃圾木块，然后根据垃圾类别设置在滑轨上的移动位置，移动到对应类别的放置位置放下垃圾卡片，完成垃圾分类功能。

6.2. 玩法步骤

注意：指令的输入需严格区分大小写，另外可按键盘“Tab”键进行关键词补齐。

1) 参照“1.教程资料\1.ArmPi Ultra 机械臂使用入门\配套手册\ArmPi Ultra 使用手册->开发环境搭建及配置”将机械臂与远程桌面工具建立连接。

2) 单击 ，启动命令栏，输入指令，按下回车，关闭自启动服务。

```
~/ .stop_ros.sh
```

```
> ~/ .stop_ros.sh
```

3) 输入运行垃圾分类的指令，按下回车。

```
ros2 launch example waste_classification_stepper.launch.py
```

```
ros2 launch example waste_classification_stepper.launch.py
```

4) 如需关闭此程序，可按下“Ctrl+C”，若玩法关闭失败可重复多次按下。

5) 玩法体验完成后，需开启 APP 服务  (若不开启则会影响后续 APP 玩法的使用)。
在终端界面输入按下回车，启动 APP 服务。

```
ros2 launch bringup bringup.launch.py
```

```
> ros2 launch bringup bringup.launch.py
```

6) APP 服务启动完成后机械臂会回到初始姿态，蜂鸣器也会“滴”一声。

6.3. 实现效果

玩法开启后，当摄像头识别到垃圾卡片后，会在画面中显示出对应的名称，不同的类别用不同颜色的矩形框出：有害垃圾为红色，可回收垃圾为蓝色，厨余垃圾为绿色，其他垃圾为灰色。

垃圾类别	卡片
hazardous_waste (有害垃圾)	Storage Battery (蓄电池)、Marker (水彩笔)、 Oral Liquid Bottle (口服液瓶)
recyclable_waste (可回收垃圾)	Plastic Bottle (塑料瓶)、Umbrella (雨伞)、 Toothbrush (牙刷)

food_waste（厨余垃圾）	Banana Peel（香蕉皮）、Ketchup（番茄酱）、 Broken Bones（碎骨头）
residual_waste（其他垃圾）	Cigarette End（烟蒂）、Plate（盘子）、 Disposable Chopsticks（一次性筷子）

6.4. 程序分析

6.4.1. Launch 文件分析

launch 文件位于：

~/ros2_ws/src/example/example/stepper/waste_classification_stepper.l

aunch.py

◆ launch_setup 函数

```
def launch_setup(context):
    compiled = os.environ['need_compile']
    if compiled == 'true':
        sdk_package_path = get_package_share_directory('sdk')
        peripherals_package_path = get_package_share_directory('peripherals')
    else:
        sdk_package_path = '/home/ubuntu/ros2_ws/src/driver/sdk'
        peripherals_package_path = '/home/ubuntu/ros2_ws/src/peripherals'

    depth_camera_launch = IncludeLaunchDescription(
        PythonLaunchDescriptionSource(
            os.path.join(peripherals_package_path, 'launch/depth_camera.launch.py')),
    )

    sdk_launch = IncludeLaunchDescription(
        PythonLaunchDescriptionSource(
            os.path.join(sdk_package_path, 'launch/armapi_ultra.launch.py')),
    )

    yolov8_node = Node(
        package='example',
        executable='yolov8_node',
        output='screen',
        parameters=[{'classes': ['BananaPeel', 'BrokenBones', 'CigaretteEnd', 'DisposableChopsticks', 'Ketchup', 'Marker', 'OralLiquidBottle', 'PlasticBottle', 'Plate', 'StorageBattery', 'Toothbrush', 'Umbrella']},
                    {'device': 'cpu', 'model': 'best_garbage', 'conf': 0.8}],
    )

    waste_classification_stepper_node = Node(
        package='example',
        executable='waste_classification_stepper',
        output='screen',
    )

    return [
        sdk_launch,
        depth_camera_launch,
        yolov8_node,
        waste_classification_stepper_node,
    ]
```

加载 peripherals 功能包的 launch/depth_camera.launch.py, 启动深度相机节点, 提供 RGB 图像和深度数据, 作为目标检测的视觉输入。加载 sdk 功能包的 launch/armapi_ultra.launch.py, 启动机械臂底层控制服务（如舵机驱动、运动学计算），为垃圾抓取动作提供硬件支持。启动 example 功能包的 yolov8_node 可执行文件, 是基于预训练模型（best_garbage）检测 12 类垃圾（如香蕉皮、塑料瓶、电池等），参数配置包括检测类别、设备（CPU）、置信度阈值（0.8），日志输出到屏幕。启动 example 功能包的 waste

_classification_stepper 可执行文件，结合 YOLOv8 的检测结果，控制步进电机驱动机械结构，将不同类型的垃圾运输到对应分类区域，完成自动化分拣。

◆ generate_launch_description 函数

```
def generate_launch_description():  
    return LaunchDescription([  
        OpaqueFunction(function = launch_setup)  
    ])
```

创建并返回 LaunchDescription 对象，通过 OpaqueFunction 调用 launch_setup 函数，作为 ROS 2 启动文件的标准入口。

◆ 主函数

```
if __name__ == '__main__':  
    ld = generate_launch_description()  
  
    ls = LaunchService()  
    ls.include_launch_description(ld)  
    ls.run()
```

创建 LaunchDescription 对象和 LaunchService 服务，将启动描述添加到启动服务并运行，实现手动启动整个系统的功能。

6.4.2. Python 文件分析

python 文件位于：

~/ros2_ws/src/example/example/stepper/include/waste_classification_stepper.py

◆ 导入功能包

```
import os
import cv2
import yaml
import time
import copy
import math
import queue
import rclpy
import signal
import threading
import numpy as np
from sdk import common
from rclpy.node import Node
from cv_bridge import CvBridge
from interfaces.msg import ObjectsInfo
from interfaces.srv import SetStringList
from std_srvs.srv import Trigger, SetBool
from sensor_msgs.msg import Image, CameraInfo
from rclpy.executors import MultiThreadedExecutor
from servo_controller_msgs.msg import ServosPosition
from ros_robot_controller_msgs.srv import MoveStepper
from rclpy.callback_groups import ReentrantCallbackGroup
from kinematics.kinematics_control import set_pose_target
from kinematics_msgs.srv import GetRobotPose, SetRobotPose
from servo_controller.bus_servo_control import set_servo_position
from app.utils import calculate_grasp_yaw, position_change_detect, pick_and_place, image_process, distortion_inverse_map
```

① `from sdk import common`: 从 SDK 中导入 common 模块, 通常包含通用的功能和工具函数。

② `from servo_controller_msgs.msg import ServosPosition`: 从 servo_controller_msgs 消息包中导入 ServosPosition 消息类型, 用于传输伺服电机的位置数据。

③ `from kinematics.kinematics_control import set_pose_target`: 从运动学控制模块中导入 set_pose_target 函数, 用于设置机器人末端执行器的目标姿态。

④ `from kinematics_msgs.srv import GetRobotPose, SetRobotPose`: 从运动学消息服务包中导入 GetRobotPose 和 SetRobotPose 服务类型, 用于获取和设置机器人的姿态。

⑤ `from servo_controller.bus_servo_control import set_servo_position`: 从伺服控制总线模块中导入 set_servo_position 函数, 用于设置伺服电机的位置。

◆ 指定目标位置

```
TARGET_POSITION = {
    'recyclable_waste': (5200),
    'hazardous_waste': (4250),
    'food_waste': (3300),
    'residual_waste': (2300)
}
```

这是一个字典，用于存储不同垃圾的目标位置。这个结构可以用来简化在机器人任务中根据颜色快速查找相应的位置。滑轨位置范围为 0(最左端)~5200(最右端)。

◆ 指定垃圾类型

```
WASTE_CLASSES = {  
    'food_waste': ('BananaPeel', 'BrokenBones', 'Ketchup'),  
    'hazardous_waste': ('Marker', 'OralLiquidBottle', 'StorageBattery'),  
    'recyclable_waste': ('PlasticBottle', 'Toothbrush', 'Umbrella'),  
    'residual_waste': ('Plate', 'CigaretteEnd', 'DisposableChopsticks'),  
}
```

这是一个字典，用于存储不同垃圾的类型。这个类型在后续的程序中用于判断放置位置。

◆ WasteClassificationNode 类初始化

```
def __init__(self, name):  
    rospy.init()  
    super().__init__(name, allow_undeclared_parameters=True, automatically_declare_parameters_from_overrides=True)  
    self.running = True  
    self.start_get_roi = True  
    self.last_position = None  
    self.roi = None  
    self.imgpts = None  
    self.count_move = 0  
    self.count_still = 0  
    self.last_position = None  
    self.extrinsic = None  
    self.intrinsic = None  
    self.distortion = None  
    self.target = None  
    self.start_transport = False  
    self.target_object_info = None  
    self.camera_type = os.environ['CAMERA_TYPE']  
    self.calibration_file = 'calibration.yaml'  
    self.config_file = 'transform.yaml'  
    self.config_path = '/home/ubuntu/ros2_ws/src/example/example/stepper/config/'  
    self.target_list = ['BananaPeel', 'BrokenBones', 'Ketchup', 'Marker', 'OralLiquidBottle', 'StorageBattery', 'PlasticBottle', 'Toothbrush', 'Umbrella', 'Plate', 'CigaretteEnd', 'DisposableChopsticks']  
  
    self.bridge = CvBridge()  
    self.joints_pub = self.create_publisher(ServoPosition, '/servo_controller', 1)  
    self.camera_info_sub = self.create_subscription(CameraInfo, '/depth_cam/rgb/camera_info', self.camera_info_callback, 1)  
    self.image_sub = self.create_subscription(Image, '/yolov8/object_image', self.image_callback, 1)  
    self.object_sub = self.create_subscription(ObjectsInfo, '/yolov8/object_detect', self.get_object_callback, 1)  
  
    # Wait for the service to start. 等待服务启动  
    self.timer_cb_group = ReentrantCallbackGroup()  
    self.client = self.create_client(Trigger, '/controller_manager/init_finish', callback_group=self.timer_cb_group)  
    self.client.wait_for_service()  
    self.get_current_pose_client = self.create_client(GetRobotPose, '/kinematics/get_current_pose', callback_group=self.timer_cb_group)  
    self.get_current_pose_client.wait_for_service()  
    self.kinematics_client = self.create_client(SetRobotPose, '/kinematics/set_pose_target', callback_group=self.timer_cb_group)  
    self.kinematics_client.wait_for_service()  
    self.start_yolov8_client = self.create_client(Trigger, '/yolov8/start', callback_group=self.timer_cb_group)  
    self.start_yolov8_client.wait_for_service()  
    self.stop_yolov8_client = self.create_client(Trigger, '/yolov8/stop', callback_group=self.timer_cb_group)  
    self.stop_yolov8_client.wait_for_service()  
    self.move_stepper_client = self.create_client(MoveStepper, '/ros_robot_controller/move_stepper', callback_group=self.timer_cb_group)  
    self.move_stepper_client.wait_for_service()  
  
    self.timer = self.create_timer(0.0, self.init_process, callback_group=self.timer_cb_group)
```

初始化节点参数（如运行状态、ROI 区域、坐标转换参数），定义垃圾类别映射（WASTE_CLASSES，将 12 类具体垃圾映射到 4 大分类）和目标放置位置（TARGET_POSITION，每类垃圾对应步进电机步数）。

◆ init_process 函数

```
def init_process(self):
    self.timer.cancel()
    req = MoveStepper.Request()
    req.steps = 0
    self.send_request(self.move_stepper_client, req)
    set_servo_position(self.joints_pub, 1.5, ((1, 210), (2, 500), (3, 110), (4, 825), (5, 600), (6, 500)))
    time.sleep(1.5)
    self.send_request(self.start_yolov8_client, Trigger.Request())
    threading.Thread(target=self.main, daemon=True).start()
    threading.Thread(target=self.transport_thread, daemon=True).start()
```

取消初始化定时器，复位步进电机，将机械臂舵机调整到初始姿态（等待 1.5 秒确保到位）。启动 YOLOv8 目标检测服务，启动主循环线程（main）和运输控制线程（transport_thread）。

◆ camera_info_callback 函数

```
def camera_info_callback(self, msg):
    self.intrinsic = np.matrix(msg.k).reshape(1, -1, 3)
    self.distortion = np.array(msg.d)
```

接收相机内参消息，提取相机矩阵（intrinsic）和畸变系数（distortion），为后续像素坐标到世界坐标的转换提供基础参数。

◆ send_request 函数

```
def send_request(self, client, msg):
    future = client.call_async(msg)
    while rclpy.ok():
        if future.done() and future.result():
            return future.result()
```

封装服务调用逻辑，发送请求后阻塞等待结果，确保机械臂运动学、步进电机等服务的通信可靠性。

◆ image_callback 函数

```
def image_callback(self, ros_image):
    cv_image = self.bridge.imgmsg_to_cv2(ros_image, "bgr8")
    bgr_image = np.array(cv_image, dtype=np.uint8)
    cv2.drawContours(bgr_image, [self.imgpts], -1, (0, 255, 255), 2, cv2.LINE_AA) #
    cv2.imshow('image', bgr_image)
    cv2.waitKey(1)
```

接收 YOLOv8 输出的标记图像，在图像上绘制 ROI 区域（感兴趣区域），实时显示检

测结果，便于调试。

◆ get_roi 函数

```
def get_roi(self):
    with open(self.config_path + self.config_file, 'r') as f:
        config = yaml.safe_load(f)

        # Convert to numpy array. 转换为 numpy 数组
        extrinsic = np.array(config['extrinsic'])
        corners = np.array(config['corners']).reshape(-1, 3)
        self.white_area_center = np.array(config['white_area_pose_world'])

    while True:
        intrinsic = self.intrinsic
        distortion = self.distortion
        if intrinsic is not None and distortion is not None:
            break
        time.sleep(0.1)

    tvec = extrinsic[:1] # Take the first row. 取第一行
    rmat = extrinsic[1:] # Take the last three rows. 取后面三行

    tvec, rmat = common.extrinsic_plane_shift(np.array(tvec).reshape((3, 1)), np.array(rmat), 0.04)
    self.extrinsic = tvec, rmat
    tvec, rmat = common.extrinsic_plane_shift(np.array(tvec).reshape((3, 1)), np.array(rmat), 0.0)
    imgpts, jac = cv2.projectPoints(corners[:-1], np.array(rmat), np.array(tvec), intrinsic, distortion)
    imgpts = np.int32(imgpts).reshape(-1, 2)
    self.imgpts = imgpts

    # 裁切出ROI区域(crop ROI region)
    x_min = min(imgpts, key=lambda p: p[0])[0] # x轴最小值(the minimum value of X-axis)
    x_max = max(imgpts, key=lambda p: p[0])[0] # x轴最大值(the maximum value of X-axis)
    y_min = min(imgpts, key=lambda p: p[1])[1] # y轴最小值(the minimum value of Y-axis)
    y_max = max(imgpts, key=lambda p: p[1])[1] # y轴最大值(the maximum value of Y-axis)
    roi = np.maximum(np.array([y_min, y_max, x_min, x_max]), 0)

    self.roi = roi
```

加载配置文件（transform.yaml），解析校准参数（如角落坐标、外参、白色区域位姿）。结合相机内参和外参，计算图像中的 ROI 区域（通过投影世界坐标到图像平面），限制检测范围，减少背景干扰，提高识别效率。

◆ get_object_world_position 函数

```
def get_object_world_position(self, position, intrinsic, extrinsic, white_area_center, height=0.03):
    projection_matrix = np.row_stack((np.column_stack((extrinsic[1], extrinsic[0])), np.array([[0, 0, 0, 1]])))
    world_pose = common.pixels_to_world([position], intrinsic, projection_matrix)[0]
    world_pose[0] = -world_pose[0]
    world_pose[1] = -world_pose[1]
    position = white_area_center[:3, 3] + world_pose
    position[2] = height

    config_data = common.get_yaml_data(os.path.join(self.config_path, self.calibration_file))
    offset = tuple(config_data['pixel']['offset'])
    scale = tuple(config_data['pixel']['scale'])
    for i in range(3):
        position[i] = position[i] * scale[i]
        position[i] = position[i] + offset[i]
    return position, projection_matrix
```

结合相机内参（intrinsic）和外参（extrinsic），通过投影矩阵计算世界坐标。

应用校准文件（calibration.yaml1）中的缩放因子和偏移量，优化坐标精度，最终返回垃圾在世界坐标系中的位置。

◆ calculate_pick_grasp_yaw 函数

```
def calculate_pick_grasp_yaw(self, position, target, target_info, intrinsic, projection_matrix):
    yaw = math.degrees(math.atan2(position[1], position[0]))
    if position[0] < 0 and position[1] < 0:
        yaw = yaw + 180
    elif position[0] < 0 and position[1] > 0:
        yaw = yaw - 180
    # 0.09x0.02
    gripper_size = [common.calculate_pixel_length(0.09, intrinsic, projection_matrix),
                    common.calculate_pixel_length(0.02, intrinsic, projection_matrix)]
    return calculate_grasp_yaw.calculate_gripper_yaw_angle(target, target_info, gripper_size, yaw)
```

根据物体世界坐标计算抓取角度（yaw），结合 gripper 尺寸和目标姿态，优化抓取方向。

◆ calculate_place_grasp_yaw 函数

```
def calculate_place_grasp_yaw(self, position, angle=0):
    yaw = math.degrees(math.atan2(position[1], position[0]))
    if position[0] < 0 and position[1] < 0:
        yaw = yaw + 180
    elif position[0] < 0 and position[1] > 0:
        yaw = yaw - 180
    yaw1 = yaw + angle
    if yaw < 0:
        yaw2 = yaw1 + 90
    else:
        yaw2 = yaw1 - 90
    yaw = yaw2
    if abs(yaw1) < abs(yaw2):
        yaw = yaw1
    yaw = 500 + int(yaw / 240 * 1000)
    return yaw
```

计算放置时的角度，确保物体正确放置到目标分类位置。

◆ transport_thread 函数

```
def transport_thread(self):
    while True:
        if self.start_transport:
            self.send_request(self.stop_yolov8_client, Trigger.Request())
            position, yaw, target = self.transport_info
            if position[0] > 0.22:
                position[2] += 0.01
            config_data = common.get_yaml_data(os.path.join(self.config_path, self.calibration_file))
            offset = tuple(config_data['kinematics']['offset'])
            scale = tuple(config_data['kinematics']['scale'])
            for i in range(3):
                position[i] = position[i] * scale[i]
                position[i] = position[i] + offset[i]

            finish = pick_and_place.pick(position, 80, yaw, 500, 0.02, self.joints_pub, self.kinematics_client)
            if finish:
                set_servo_position(self.joints_pub, 1.0, ((6, 500),))
                time.sleep(1.0)

                # 创建 MoveStepper 请求对象并设置步数
                stepper_position = TARGET_POSITION[target]
                req = MoveStepper.Request() # 正确创建 MoveStepper 请求对象
                req.steps = stepper_position # 设置步进电机的步数

                # 发送请求给 MoveStepper 服务
                self.send_request(self.move_stepper_client, req)
                stepper_time = stepper_position / 1000 # 计算所需时间
                time.sleep(stepper_time)

                # 设置放置位置
                position = [0.25, 0, 0.02]
                yaw = self.calculate_place_grasp_yaw(position, 0)

                # 处理位置和角度调整
                config_data = common.get_yaml_data(os.path.join(self.config_path, self.calibration_file))
                offset = tuple(config_data['kinematics']['offset'])
                scale = tuple(config_data['kinematics']['scale'])
                angle = math.degrees(math.atan2(position[1], position[0]))

                if angle > 45:
                    position = [position[0] * scale[1], position[1] * scale[0], position[2] * scale[2]]
                    position = [position[0] - offset[1], position[1] + offset[0], position[2] + offset[2]]
                elif angle < -45:
                    position = [position[0] * scale[1], position[1] * scale[0], position[2] * scale[2]]
                    position = [position[0] + offset[1], position[1] - offset[0], position[2] + offset[2]]
                else:
                    position = [position[0] * scale[0], position[1] * scale[1], position[2] * scale[2]]
                    position = [position[0] + offset[0], position[1] + offset[1], position[2] + offset[2]]

                # 执行放置操作
                finish = pick_and_place.place(position, 80, yaw, 200, self.joints_pub, self.kinematics_client)
                if finish:
                    set_servo_position(self.joints_pub, 1.5, ((1, 210), (2, 500), (3, 110), (4, 825), (5, 600), (6, 500)))
                    time.sleep(1.5)

                    # 再次发送请求，步进电机返回原位
                    stepper_position = TARGET_POSITION[target]
                    req = MoveStepper.Request() # 正确创建 MoveStepper 请求对象
                    req.steps = -stepper_position # 让步进电机回到原位
                    self.send_request(self.move_stepper_client, req)
                    stepper_time = stepper_position / 1000 # 计算所需时间
                    time.sleep(stepper_time)

                    # 重新启动 yolov8
                    self.send_request(self.start_yolov8_client, Trigger.Request())
                    self.target = None
                    self.start_transport = False
                else:
                    time.sleep(0.1)
```

当 start_transport 为 True 时，执行“抓取 - 运输 - 放置”全流程：停止 YOLOv8 检测，避免抓取时干扰。调用 pick_and_place.pick 控制机械臂抓取垃圾。通过步进电机服务（move_stepper）将垃圾运输到对应分类区域（根据垃圾类别从 TARGET_POS

ITION 获取步数)。调用 pick_and_place.place 控制机械臂放置垃圾。步进电机复位，机械臂回到初始姿态，重新启动 YOLOv8，重置状态变量。

◆ main 方法

```
def main(self):
    while self.running:
        if self.start_get_roi:
            self.get_roi()
            self.start_get_roi = False
        roi = self.roi
        if self.target_object_info is not None:
            target_object_info = copy.deepcopy(self.target_object_info)
            center = target_object_info[0][2]
            if self.camera_type == 'USB_CAM':
                x, y = distortion_inverse_map.undistorted_to_distorted_pixel(center[0], center[1], self.intrinsic, self.distortion)
                center = (x, y)
            intrinsic = self.intrinsic
            self.target_object_info = None

        if roi[2] < center[0] < roi[3] and roi[0] < center[1] < roi[1]:
            position, projection_matrix = self.get_object_world_position(target_object_info[0][2], intrinsic, self.extrinsic, self.white_area_center, 0.04)
            result = self.calculate_pick_grasp_yaw(position, target_object_info[0], target_object_info[1], intrinsic, projection_matrix)
            if result is not None:
                if self.last_position is not None:
                    e_distance = round(math.sqrt(pow(self.last_position[0] - position[0], 2)) + math.sqrt(pow(self.last_position[1] - position[1], 2)), 5)
                    # self.get_logger().info(f'e_distance')
                    if e_distance <= 0.002: # Euclidean distance less than 2mm to prevent grasping while the object is still moving. 欧式距离小于2mm, 防止物体还在移动时就去夹取了
                        self.count_move = 0
                        self.count_still += 1
                    else:
                        self.count_move += 1
                        self.count_still = 0
                    if self.count_move > 10:
                        if target_object_info[0][0] in self.target_list:
                            self.target_list.remove(target_object_info[0][0])

                if self.count_still > 4:
                    self.count_still = 0
                    self.count_move = 0
                    for k, v in WASTE_CLASSES.items():
                        if target_object_info[0][0] in v:
                            waste_category = k
                            break
                    yaw = 500 + int(result[0] / 240 * 1000)
                    self.transport_info = [position, yaw, waste_category]
                    # self.get_logger().info(f'{self.transport_info}')
                    self.start_transport = True
                self.last_position = position
```

调用 get_roi 初始化 ROI 区域,持续监控目标检测结果。当检测到垃圾且位置稳定(连续 4 帧位移小于 0.002 米)时,通过 WASTE_CLASSES 映射垃圾类别,计算抓取位置和角度,触发运输流程(start_transport = True)。若垃圾位置持续移动(超过 10 帧),移除该目标,避免无效抓取。

◆ get_object_callback 函数

```
def get_object_callback(self, msg):
    objects = msg.objects
    local_target_object_info = None
    local_objects_list = []
    local_object_info = None
    class_name = None

    target_object_info = self.target_object_info
    if not self.start_transport and objects and target_object_info is None:
        for i in objects:
            # Calculate the Angle of the object. 计算对象的角度
            if i.angle < 0:
                i.angle = 90 - abs(i.angle)

            target = [i.class_name, 0, (int(i.box[0]), int(i.box[1])), (int(i.box[2]), int(i.box[3])), i.angle]

            # If the target object is found. 如果找到目标对象
            if i.class_name in self.target_list:
                if local_object_info is None:
                    local_object_info = target

                if local_object_info[0] == i.class_name:
                    class_name = i.class_name
                    local_object_info = target

            local_objects_list.append(target)

        if class_name is not None:
            local_target_object_info = [local_object_info, local_objects_list]
    if not self.start_transport and target_object_info is None:
        self.target_object_info = copy.deepcopy(local_target_object_info)
```

订阅 YOLOv8 的目标检测消息 (ObjectsInfo)，提取垃圾的类别、坐标、角度等信息，筛选出 target_list 中的有效垃圾，为 main 线程提供目标数据。

◆ 主函数

```
343 def main():
344     node = WasteClassificationNode('waste_classification')
345     executor = MultiThreadedExecutor()
346     executor.add_node(node)
347     executor.spin()
348     node.destroy_node()
349     rclpy.shutdown()
```

启动和运行垃圾分类 ROS2 节点的入口函数。创建 WasteClassificationNode 实例，使用 MultiThreadedExecutor 支持多线程处理，启动节点运行并在结束时正确清理资源。

7. 标签分拣

7.1. 玩法简介

本节课通过摄像头识别标签，然后通过识别的标签信息以及位置，通过逆运动学求解的结果控制机械臂抓取标签木块，然后驱动滑轨到达指定位置，将标签木块放置到对应位置。

AprilTag 作为一种视觉定位标志符，类似于二维码或者条形码。

这里我们先订阅摄像头节点发布的信息，获取实时图像数据，将 RGB 颜色空间转换为灰度图，再读取相机内置参数。

接着，通过调用 AprilTag 库文件函数对获取的图像进行 AprilTag 检测，识别出标签 ID，获取到标签码四个角点坐标及中心点坐标信息。

计算识别的坐标信息得到木块的位置坐标，通过逆运动学来计算目标位置处的各个舵机的角度，再控制机械臂到达目标位置对标签木块进行抓取，并驱动滑轨搬运到对应 ID 的位置将木块放置。

7.2. 玩法步骤

注意：指令的输入需严格区分大小写，另外可按键盘“Tab”键进行关键词补齐。

1) 参照“1.教程资料\1.ArmPi Ultra 机械臂使用入门\配套手册\ArmPi Ultra 使用手册->开发环境搭建及配置”将机械臂与远程桌面工具建立连接。

2) 单击 ，启动命令栏，输入指令，按下回车，关闭自启动服务。

```
~/ .stop_ros.sh
```

```
> ~/ .stop_ros.sh
```

3) 打开一个新的命令栏，输入运行标签分拣的指令，按下回车。

```
ros2 launch example tag_sorting_stepper.launch.py
```

```
ros2 launch example tag_sorting_stepper.launch.py
```

4) 如需关闭此程序，可按下“Ctrl+C”，若玩法关闭失败可重复多次按下。

5) 玩法体验完成后，需开启 APP 服务  (若不开启则会影响后续 APP 玩法的使用)。
在终端界面输入按下回车，启动 APP 服务。

```
ros2 launch bringup bringup.launch.py
```

```
> ros2 launch bringup bringup.launch.py
```

6) APP 服务启动完成后机械臂会回到初始姿态，蜂鸣器也会“滴”一声。

7.3. 实现效果

开启玩法后，将标签木块放置在机械臂的视觉识别区，机械臂将会按标签 ID1、2、3 的顺序对木块进行抓取，并放置到对应的位置。

7.4. 程序分析

7.4.1. Launch 文件分析

launch 文件位于：

~/ros2_ws/src/example/example/stepper/tag_sorting_stepper.launch.py

◆ launch_setup 函数

```
def launch_setup(context):
    compiled = os.environ['need_compile']
    if compiled == 'True':
        sdk_package_path = get_package_share_directory('sdk')
        peripherals_package_path = get_package_share_directory('peripherals')
    else:
        sdk_package_path = '/home/ubuntu/ros2_ws/src/driver/sdk'
        peripherals_package_path = '/home/ubuntu/ros2_ws/src/peripherals'

    depth_camera_launch = IncludeLaunchDescription(
        PythonLaunchDescriptionSource(
            os.path.join(peripherals_package_path, 'launch/depth_camera.launch.py')),
    )

    sdk_launch = IncludeLaunchDescription(
        PythonLaunchDescriptionSource(
            os.path.join(sdk_package_path, 'launch/arduino_ultra.launch.py')),
    )

    tag_sorting_stepper_node = Node(
        package='example',
        executable='tag_sorting_stepper',
        output='screen',
    )

    return [
        sdk_launch,
        depth_camera_launch,
        tag_sorting_stepper_node,
    ]
```

加载 peripherals 功能包的 `launch/depth_camera.launch.py`, 启动深度相机节点, 提供 RGB 图像和深度数据, 作为标签识别的视觉输入 (如二维码、条形码或特定标记)。加载 sdk 功能包的 `launch/arm_pi_ultra.launch.py`, 启动机械臂底层控制服务 (如舵机驱动、运动学计算), 为抓取带标签的物体提供硬件支持。启动 example 功能包的 `tag_sorting_stepper` 可执行文件, 核心功能是结合相机识别的标签信息 (如标签类型、ID), 控制步进电机驱动机械结构, 将不同标签的物体运输到对应分类区域, 完成自动化分拣, 日志输出到屏幕。

◆ generate_launch_description 函数

```
def generate_launch_description():  
    return LaunchDescription([  
        OpaqueFunction(function = launch_setup)  
    ])
```

创建并返回 `LaunchDescription` 对象, 通过 `OpaqueFunction` 调用 `launch_setup` 函数, 作为 ROS 2 启动文件的标准入口。

◆ 主函数

```
if __name__ == '__main__':  
    ld = generate_launch_description()  
  
    ls = LaunchService()  
    ls.include_launch_description(ld)  
    ls.run()
```

创建 `LaunchDescription` 对象和 `LaunchService` 服务, 将启动描述添加到启动服务并运行, 实现手动启动整个系统的功能。

7.4.2. Python 文件分析

python 文件位于:

`~/ros2_ws/src/example/example/stepper/include/tag_sorting_stepper.py`

◆ 导入功能包

```
import os
import cv2
import yaml
import time
import math
import queue
import rclpy
import threading
import numpy as np
from sdk import common
from rclpy.node import Node
from app.common import Heart
from cv_bridge import CvBridge
from dt_apriltags import Detector
from std_srvs.srv import Trigger, SetBool
from sensor_msgs.msg import Image, CameraInfo
from rclpy.executors import MultiThreadedExecutor
from interfaces.srv import SetStringBool
from ros_robot_controller_msgs.srv import MoveStepper
from servo_controller_msgs.msg import ServosPosition
from rclpy.callback_groups import ReentrantCallbackGroup
from kinematics.kinematics_control import set_pose_target
from kinematics_msgs.srv import GetRobotPose, SetRobotPose
from kinematics.kinematics_control import set_joint_value_target
from servo_controller.bus_servo_control import set_servo_position
from app.utils import calculate_grasp_yaw, position_change_detect, pick_and_place, image_process, distortion_inverse_map
```

- ① **sdk.common**: 自定义 SDK 中的通用工具。
- ② **app.common.Heart**: 应用程序中的心跳功能模块, 用于监控系统状态。
- ③ **dt_apriltags.Detector**: AprilTag 检测器, 用于识别和跟踪 AprilTag 标签。
- ④ **servo_controller_msgs.msg.ServosPosition**: 自定义 ROS 消息, 用于表示伺服电机的位置。
- ⑤ **kinematics.kinematics_control.set_pose_target**: 运动学控制模块中的设置姿态目标函数。
- ⑥ **servo_controller.bus_servo_control.set_servo_position**: 伺服控制模块中的设置伺服位置函数。

◆ 指定目标位置

```
TARGET_POSITION = {
    'target_1': (2600),
    'target_2': (4000),
    'target_3': (5200),
}
```

这是一个字典, 用于存储不同标签的目标位置。这个结构可以用来简化在机器人任务中根据标签快速查找相应的位置。滑轨位置范围为 0(最左端)~5200(最右端)。

◆ TagSorting 类

```

class TagSorting(Node):
    def __init__(self, name):
        rclpy.init()
        super().__init__(name, allow undeclared parameters=True, automatically_declare_parameters_from_overrides=True)
        self.tag_size = 0.025
        self.start_get_roi = True
        self.last_position = None
        self.imgpts = None
        self.roi = None
        self.count_move = 0
        self.count_still = 0
        self.last_position = None
        self.start_transport = False
        self.extrinsic = None
        self.intrinsic = None
        self.distortion = None
        self.target = None
        self.target_miss_count = 0
        self.camera_type = os.environ['CAMERA_TYPE']
        self.calibration_file = 'calibration.yaml'
        self.config_file = 'transform.yaml'
        self.config_path = "/home/ubuntu/ros2_ws/src/example/example/stepper/config/"

        self.image_queue = queue.Queue(maxsize=2)
        self.lock = threading.RLock()
        self.bridge = CvBridge() # Used for the conversion between ROS Image messages and OpenCV images. 用于ROS Image消息与OpenCV图像之间的转换
        self.at_detector = Detector(searchpath=['apriltags'],
                                    families='tag36h11',
                                    nthreads=4,
                                    quad_decimate=1.0,
                                    quad_sigma=0.0,
                                    refine_edges=1,
                                    decode_sharpening=0.25,
                                    debug=0)

```

```

self.joints_pub = self.create_publisher(ServosPosition, '/servo_controller', 1)
self.image_sub = self.create_subscription(Image, '/depth_cam/rgb/image_raw', self.image_callback, 1)
self.camera_info_sub = self.create_subscription(CameraInfo, '/depth_cam/rgb/camera_info', self.camera_info_callback, 1)

self.timer_cb_group = ReentrantCallbackGroup()
self.client = self.create_client(Trigger, '/controller_manager/init_finish', callback_group=self.timer_cb_group)
self.client.wait_for_service()
self.get_current_pose_client = self.create_client(GetRobotPose, '/kinematics/get_current_pose', callback_group=self.timer_cb_group)
self.get_current_pose_client.wait_for_service()
self.kinematics_client = self.create_client(SetRobotPose, '/kinematics/set_pose_target', callback_group=self.timer_cb_group)
self.kinematics_client.wait_for_service()
self.move_stepper_client = self.create_client(MoveStepper, '/ros_robot_controller/move_stepper', callback_group=self.timer_cb_group)
self.move_stepper_client.wait_for_service()
self.timer = self.create_timer(0.0, self.init_process, callback_group=self.timer_cb_group)

```

用于初始化标签分拣节点。该函数设置了 AprilTag 相关参数（标签尺寸、检测器配置），初始化状态变量和计数器，配置步进电机控制器，创建图像队列和线程锁，建立 ROS2 的发布者、订阅者和服务客户端，并通过定时器启动初始化流程。其中 AprilTag 检测器配置了 tag36h11 系列等参数以提高检测精度。

◆ init_process 函数

```

def init_process(self):
    self.timer.cancel()
    req = MoveStepper.Request()
    req.steps = 0
    self.send_request(self.move_stepper_client, req)
    set_servo_position(self.joints_pub, 1.5, ((1, 210), (2, 500), (3, 110), (4, 825), (5, 600), (6, 500)))
    time.sleep(1.5)
    threading.Thread(target=self.main, daemon=True).start()
    threading.Thread(target=self.transport_thread, daemon=True).start()

```

取消初始化定时器，复位步进电机（步数设为 0），将机械臂舵机调整到初始姿态（等待 1.5 秒确保到位）。启动主循环线程（main）和运输控制线程（transport_thread），准备开始标签检测与分拣。

◆ send_request 函数

```
def send_request(self, client, msg):
    future = client.call_async(msg)
    while rclpy.ok():
        if future.done() and future.result():
            return future.result()
```

封装服务调用逻辑，发送请求后阻塞等待结果，确保机械臂运动学、步进电机等服务的通信可靠性。

◆ camera_info_callback 函数

```
def camera_info_callback(self, msg):
    self.intrinsic = np.matrix(msg.k).reshape(1, -1, 3)
    self.distortion = np.array(msg.d)
```

接收相机内参消息，提取相机矩阵（intrinsic）和畸变系数（distortion），为后续像素坐标到世界坐标的转换提供基础参数。

◆ image_callback 函数

```
def image_callback(self, ros_image):
    # Convert ros format images to opencv format. 将ros格式图像转换为opencv格式
    cv_image = self.bridge.imgmsg_to_cv2(ros_image, "bgr8")
    bgr_image = np.array(cv_image, dtype=np.uint8)

    if self.image_queue.full():
        # If the queue is full, discard the oldest image. 如果队列已满，丢弃最旧的图像
        self.image_queue.get()
    # Put the image into the queue. 将图像放入队列
    self.image_queue.put(bgr_image)
```

将 ROS 图像消息（sensor_msgs/Image）转换为 OpenCV 格式（BGR），存入图像队列（队列满时丢弃旧图像），供主循环处理标签检测。

◆ get_roi 函数

```
def get_roi(self):
    with open(self.config_path + self.config_file, 'r') as f:
        config = yaml.safe_load(f)

        # Convert to numpy array. 转换为 numpy 数组
        extrinsic = np.array(config['extrinsic'])
        corners = np.array(config['corners']).reshape(-1, 3)
        self.white_area_center = np.array(config['white_area_pose_world'])

    while True:
        intrinsic = self.intrinsic
        distortion = self.distortion
        if intrinsic is not None and distortion is not None:
            break
        time.sleep(0.1)

    tvec = extrinsic[:1] # Take the first row. 取第一行
    rmat = extrinsic[1:] # Take the last three rows. 取后面三行

    tvec, rmat = common.extrinsic_plane_shift(np.array(tvec).reshape((3, 1)), np.array(rmat), 0.03)
    self.extrinsic = tvec, rmat
    tvec, rmat = common.extrinsic_plane_shift(np.array(tvec).reshape((3, 1)), np.array(rmat), 0.00)
    imgpts, jac = cv2.projectPoints(corners[:-1], np.array(rmat), np.array(tvec), intrinsic, distortion)
    imgpts = np.int32(imgpts).reshape(-1, 2)
    self.imgpts = imgpts
    # Crop ROI region 裁切出ROI区域
    x_min = min(imgpts, key=lambda p: p[0])[0] # The minimum value of the X-axis. x轴最小值
    x_max = max(imgpts, key=lambda p: p[0])[0] # The maximum value of the X-axis. x轴最大值
    y_min = min(imgpts, key=lambda p: p[1])[1] # The minimum value of the Y-axis. y轴最小值
    y_max = max(imgpts, key=lambda p: p[1])[1] # The maximum value of the Y-axis. y轴最大值
    roi = np.maximum(np.array([y_min, y_max, x_min, x_max]), 0)

    self.roi = roi
```

加载配置文件（transform.yaml），解析校准参数（如角落坐标、外参、白色区域位姿）。结合相机内参和外参，将世界坐标系中的参考点投影到图像平面，计算 ROI 区域（感兴趣区域），限制检测范围以减少背景干扰，提高标签识别效率。

◆ get_object_world_position 函数

```
def get_object_world_position(self, position, intrinsic, extrinsic, white_area_center, height=0.03):
    projection_matrix = np.row_stack((np.column_stack((extrinsic[1], extrinsic[0])), np.array([[0, 0, 0, 1]])))
    world_pose = common.pixels_to_world([position], intrinsic, projection_matrix)[0]
    world_pose[0] = -world_pose[0]
    world_pose[1] = -world_pose[1]
    position = white_area_center[:3, 3] + world_pose
    position[2] = height
    config_data = common.get_yaml_data(os.path.join(self.config_path, self.calibration_file))
    offset = tuple(config_data['pixel']['offset'])
    scale = tuple(config_data['pixel']['scale'])
    for i in range(3):
        position[i] = position[i] * scale[i]
        position[i] = position[i] + offset[i]
    return position, projection_matrix
```

结合相机内参（intrinsic）和外参（extrinsic），通过投影矩阵计算标签的世界坐标。应用校准文件（calibration.yaml）中的缩放因子和偏移量优化坐标精度，最终返回标签在世界坐标系中的三维位置，为机械臂抓取提供准确坐标。

◆ calculate_pick_grasp_yaw 函数

```
def calculate_pick_grasp_yaw(self, position, target, target_info, intrinsic, projection_matrix):
    yaw = math.degrees(math.atan2(position[1], position[0]))
    if position[0] < 0 and position[1] < 0:
        yaw = yaw + 180
    elif position[0] < 0 and position[1] > 0:
        yaw = yaw - 180
    # 0.09x0.02
    gripper_size = [common.calculate_pixel_length(0.09, intrinsic, projection_matrix),
                    common.calculate_pixel_length(0.02, intrinsic, projection_matrix)]

    return calculate_grasp_yaw.calculate_gripper_yaw_angle(target, target_info, gripper_size, yaw)
```

根据物体世界坐标计算抓取角度（yaw），结合机械爪尺寸和目标姿态，优化抓取方向，确保稳定抓取标签。

◆ calculate_place_grasp_yaw 函数

```
def calculate_place_grasp_yaw(self, position, angle=0):
    yaw = math.degrees(math.atan2(position[1], position[0]))
    if position[0] < 0 and position[1] < 0:
        yaw = yaw + 180
    elif position[0] < 0 and position[1] > 0:
        yaw = yaw - 180
    yaw1 = yaw + angle
    if yaw < 0:
        yaw2 = yaw1 + 90
    else:
        yaw2 = yaw1 - 90

    yaw = yaw2
    if abs(yaw1) < abs(yaw2):
        yaw = yaw1
    yaw = 500 + int(yaw / 240 * 1000)

    return yaw
```

计算放置时的角度，确保标签被准确放置到目标分类区域。

◆ transport_thread 函数

```

def transport_thread(self):
    while True:
        if self.start_transport:
            position, yaw, target = self.transport_info
            if position[0] > 0.22:
                position[2] += 0.01
            config_data = common.get_yaml_data(os.path.join(self.config_path, self.calibration_file))
            offset = tuple(config_data['kinematics']['offset'])
            scale = tuple(config_data['kinematics']['scale'])
            for i in range(3):
                position[i] = position[i] * scale[i]
                position[i] = position[i] + offset[i]

            finish = pick_and_place.pick(position, 80, yaw, 540, 0.02, self.joints_pub, self.kinematics_client)
            if finish:
                set_servo_position(self.joints_pub, 1.5, ((1, 210), (2, 500), (3, 110), (4, 825), (5, 600), (6, 500)))
                time.sleep(1.5)
                stepper_position = TARGET_POSITION[str(self.target[0])]
                req = MoveStepper.Request()
                req.steps = stepper_position
                self.send_request(self.move_stepper_client, req)
                stepper_time = stepper_position/1000 # Calculate the required time. 计算需要的时间
                time.sleep(stepper_time)

                position = [0.25, 0, 0.015] # Set the placement position. 设置放置位置

                yaw = self.calculate_place_grasp_yaw(position, 0)
                config_data = common.get_yaml_data(os.path.join(self.config_path, self.calibration_file))
                offset = tuple(config_data['kinematics']['offset'])
                scale = tuple(config_data['kinematics']['scale'])
                angle = math.degrees(math.atan2(position[1], position[0]))
                if angle > 45:
                    # self.get_logger().info(f'1:{position}')
                    position = [position[0] * scale[1], position[1] * scale[0], position[2] * scale[2]]
                    position = [position[0] - offset[1], position[1] + offset[0], position[2] + offset[2]]
                elif angle < -45:
                    # self.get_logger().info(f'2:{position}')
                    position = [position[0] * scale[1], position[1] * scale[0], position[2] * scale[2]]
                    position = [position[0] + offset[1], position[1] - offset[0], position[2] + offset[2]]
                else:
                    # self.get_logger().info(f'3:{position}')
                    position = [position[0] * scale[1], position[1] * scale[0], position[2] * scale[2]]
                    position = [position[0] + offset[0], position[1] + offset[1], position[2] + offset[2]]

                # self.get_logger().info(f'{position}')
                finish = pick_and_place.place(position, 80, yaw, 200, self.joints_pub, self.kinematics_client)

```

```

            if finish:
                set_servo_position(self.joints_pub, 1.5, ((1, 210), (2, 500), (3, 110), (4, 825), (5, 600), (6, 500)))
                time.sleep(1.5)
                req = MoveStepper.Request()
                req.steps = -stepper_position
                self.send_request(self.move_stepper_client, req)
                stepper_time = stepper_position/1000 # 计算需要的时间
                time.sleep(stepper_time)
                self.target = None
                self.start_transport = False
            else:
                time.sleep(0.1)

```

当 start_transport 为 True 时，执行“抓取 - 运输 - 放置”全流程：调用 pick_and_place.pick 控制机械臂抓取标签（基于计算的世界坐标和角度）。通过步进电机服务（move_stepper）将标签运输到对应位置（根据标签 ID 从 TARGET_POSITION 获取步数）。调用 pick_and_place.place 控制机械臂放置标签。步进电机复位，机械臂回到初始姿态，重置状态变量（start_transport=False）。

◆ main 函数

```

def main(self):
    while True:
        try:
            bgr_image = self.image_queue.get(block=True, timeout=1)
        except queue.Empty:
            continue
        if self.start_get_roi:
            self.get_roi()
            self.start_get_roi = False
        roi = self.roi.copy()
        intrinsic = self.intrinsic
        target_info = []
        if not self.start_transport:
            tags = self.at_detector.detect(cv2.cvtColor(bgr_image, cv2.COLOR_RGB2GRAY), True, (intrinsic[0,0], intrinsic[1,1], intrinsic[0,2], intrinsic[1,2]), self.tag_size)
            if len(tags) > 0 and roi[0] < tags[0].center[1] and roi[1] < tags[0].center[2] < tags[0].center[0] < roi[3]:
                common.draw_tags(bgr_image, tags, corners_color=(0, 0, 255), center_color=(0, 255, 0))
                index = 0
                for tag in tags:
                    corners = tag.corners.astype(int)
                    rect = cv2.minAreaRect(np.array(tag.corners).astype(np.float32))
                    # rect includes center point, (width, height), rotation Angle. rect 包含 (中心点, (宽度, 高度), 旋转角度)
                    (center, (width, height), angle) = rect
                    index += 1
                    target_info.append(['tagId', tag.tag_id, index, (int(center[0]), int(center[1])), (int(width), int(height)), angle])
                    cv2.putText(bgr_image, "%d" % tag.tag_id, (int(tag.center[0]) - (7 * len("%d" % tag.tag_id))), int(tag.center[1]-10), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (255, 255, 0), 2)
            target_miss = True
            for target in target_info:
                if self.target is not None: # If there is already a target, skip the other objects directly. 如果已经有了目标, 其他物体就直接跳过
                    if self.target[0] != target[0] or self.target[1] != target[1]:
                        continue
                    else:
                        target_miss = False
                        self.target = target
                position, projection_matrix = self.get_object_world_position(target[2], intrinsic, self.extrinsic, self.white_area_center, 0.03)
                result = self.calculate_pick_grasp_yaw(position, target, target_info, intrinsic, projection_matrix)
                if result is not None and self.target is None:
                    self.target = target
                    break
                if self.last_position is not None and self.target is not None:
                    e_distance = round(math.sqrt(pow(self.last_position[0] - position[0], 2)) + math.sqrt(pow(self.last_position[1] - position[1], 2)), 5)
                    # self.get_logger().info(f'e_distance: {e_distance}')
                    if e_distance <= 0.001: # Euclidean distance less than 1mm to prevent grasping while the object is still moving. 欧式距离小于1mm, 防止物体还在移动时就去夹取了
                        cv2.line(bgr_image, result[1][0], result[1][1], (255, 255, 0), 2, cv2.LINE_AA)
                        self.count_move = 0
                        self.count_still += 1
                    else:

```

```

                        self.count_move += 1
                        self.count_still = 0

                if self.count_move > 10:
                    self.target = None
                if self.count_still > 20:
                    self.count_still = 0
                    self.count_move = 0
                    self.target = target
                    yaw = 500 + int(result[0] / 240 * 1000)
                    self.transport_info = [position, yaw, target]
                    self.start_transport = True
                    self.last_position = position
            if target_miss:
                self.target_miss_count += 1
            if self.target_miss_count > 10:
                self.target_miss_count = 0
                self.target = None
            cv2.drawContours(bgr_image, [self.imgpts], -1, (0, 255, 255), 2, cv2.LINE_AA) # Draw rectangle 绘制矩形
        if bgr_image is not None:
            cv2.imshow('result', bgr_image)
            cv2.waitKey(1)

```

从图像队列获取图像，通过 dt_apriltags 检测器识别 AprilTag，提取标签 ID、中心坐标、角度等信息。若标签连续 20 帧位移小于 0.001 米（视为稳定），触发运输流程（start_transport=True）；若位置持续移动超过 10 帧，重置目标。实时显示标记后的图像（绘制标签轮廓、中心、ROI 区域），便于调试与监控。

◆ 主函数

```
def main():  
    node = TagSorting('tag_sorting')  
    executor = MultiThreadedExecutor()  
    executor.add_node(node)  
    executor.spin()  
    node.destroy_node()  
    rclpy.shutdown()
```

启动和运行标签分拣 ROS2 节点的入口函数。创建 TagSorting 实例，使用 MultiThreadedExecutor 支持多线程处理，启动节点运行并在结束时正确清理资源。